Embedded Systems

Computer & Systems Engineering department

4th year students

2024-2025

University Book Publishing and Distribution Apparatus
Helwan University

2024_2025

Embedded Systems

Prepared by:

Dr. Hossam Eldin Ibrahim Ali

Prof. Medhat Awadallah

Publisher

University Book Publishing and Distribution Authority

Helwan University

2024_2025

Introducing the book

This book is prepared for students to understand one of the most important processor families in embedded systems. The book is prepared so that student will gain the following main concepts about embedded systems based on ARM processor.

1. **Understanding the Arm Architecture**:
   - The Arm architecture is widely used globally, powering billions of devices. It consists of three main profiles: Arm A (application processors), Arm R (real-time processors), and Arm M (microcontrollers).
2. **Programming Model**:
   - Learn about the processor's programming model, which includes registers, modes, and states. Understand how the processor operates and transitions between different execution states.
3. **Assembly Language**:
   - Dive into ARM assembly language. This involves understanding instructions, addressing modes, arithmetic operations, logical operators, branches, loops, and subroutine calls. As you progress, you'll gain the ability to write assembly code that interacts with hardware devices.
4. **C Programming with ARM**:
   - Interface assembly code with C programs. Learn how to write efficient and optimized code by combining assembly and high-level C language constructs.
5. **Toolchains and Emulators**:
   - Explore ARM toolchains for cross-compiling and linking programs. Familiarize yourself with system emulators that allow you to run and test your ARM programs virtually.
6. **I/O Device interfacing**:
   - develop simple I/O device drive functions. These functions enable communication between your software and external hardware components.

Chapter 1 Introduction to embedded systems

Main points

- What is an embedded system?
- Benefits of Embedded Computer Systems
- Application examples
    - Bike Computers and Embedded Systems
    - Engine Control Units (ECUs) and Their Demands
- Embedding a computer

What is an embedded system?

- *Application-specific* computer system
- Component of a larger system
- Interacts with its environment
- Often has *real-time* computing constraints

- An embedded system (ES) is a specialized computer system with chip embedded hardware. There is no standard definitions for an ES, but principally it can be defined as a hybrid (pre-)processing and computing system, with a task specialized (internal) design which interacts with its environment and usually has timing constraints. Usually ESs are designed because a repeating task has to be performed, either periodically or spontaneously, with low cost and power, higher performance, etc.
- A real time system is the system that guarantees output within a defined period of time or interval.

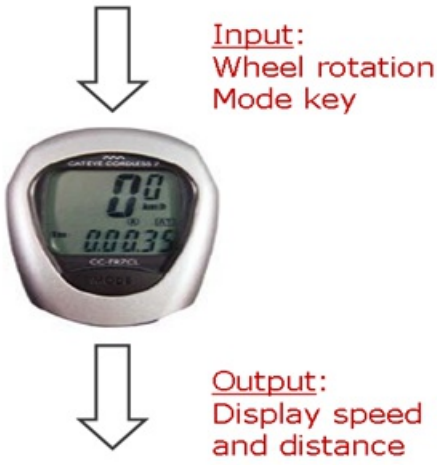
Benefits of Embedded Computer Systems

- Greater performance and efficiency
  - Software makes it possible to provide sophisticated control
  - Integrated functions often more efficient than external ones
- Lower costs
  - Less expensive components can be used
  - Manufacturing, operating, and maintenance costs reduced
- More features
  - Many not possible or practical with other approaches
- Better dependability/security
  - Adaptive system which can compensate for failures
  - Better diagnostics to improve repair time
- Potential for distributed system design
  - Multiple processors communicating across a network can lower parts and assembly costs and improve reliability

Application examples

- Simple control: microwave oven front panel
- Canon EOS 3 has three microprocessors.
- 32-bit RISC CPU runs auto-focus and eye control systems.
    - Digital TV: programmable CPUs + hardwired logic.
- Smart phone: keyboard, communications, games, app's
- Internet of Things (IoT) - distributed sensors/controllers
- Vehicle control (automotive, aerospace, etc.)
- Industrial process control (nuclear power plant)

Bike Computers and Embedded Systems



A bike computer is a nifty little device that enhances your cycling experience by providing real-time information about your ride. Whether you're a casual cyclist or a serious racer, a bike computer can be your trusty companion. Let's break down the key points from your description:

1. **Purpose of a Bike Computer:**
   - A bike computer serves as an onboard information center for cyclists. It tracks various parameters related to your ride, such as speed, distance traveled, time elapsed, and sometimes even heart rate.
   - Cyclists use this data for training, navigation, and overall enjoyment.
2. **Embedded System Characteristics:**
   - An embedded system is a specialized computer system designed for a specific purpose. In the case of a bike computer, it's a small-scale embedded system.
   - Key characteristics include:
     - **Size and Weight:** Bike computers need to be compact and lightweight. After all, you don't want a bulky device weighing down your handlebars.
     - **Cost Efficiency:** Keeping costs low is crucial, especially for mass-produced bike computers.
     - **Power Efficiency:** Since bike computers run on batteries, optimizing power consumption is essential for longer battery life.
     - **Real-Time Constraints:** Bike computers operate in real time. They need to respond quickly to sensor inputs (like wheel rotations) and provide timely updates to the user.
3. **Sensor Input:**
   - The primary sensor in a bike computer is the wheel rotation sensor. Most commonly, this sensor detects wheel movement magnetically (using a magnet on the wheel and a sensor on the frame).
   - As the wheel rotates, the sensor generates pulses. By counting these pulses, the bike computer calculates speed, distance, and other metrics.
4. **Processing Requirements:**
   - Given the real-time nature of the system, the bike computer must process sensor data promptly.
   - For a non-professional cyclist, measuring speeds up to 60 km/h (which translates to 16.7 m/s) is sufficient. Considering a racing bike's wheel circumference (around 2 meters), this corresponds to approximately 8–9 rotations per second.
   - Therefore, the system should handle a frequency of at least 10 Hz without complex processing algorithms.

5. **Microcontroller Selection:**
    - To keep costs low and power efficiency high, a simple microcontroller suffices.
    - The microcontroller's main tasks include:
        - Counting sensor pulses.
        - Converting pulse counts to speed and distance.
        - Displaying relevant information on the screen (such as an LCD display).
    - Common choices might include low-power microcontrollers from the AVR or ARM Cortex-M families.
6. **User Interface:**
    - Bike computers typically have a small LCD or LED display to show speed, distance, and other relevant data.
    - Some advanced models may include additional features like GPS navigation, heart rate monitoring, and wireless connectivity.
7. **Startup Behavior:**
    - The bike computer should start recording data as soon as it detects wheel rotation (between two sensor impulses).
    - This ensures accurate tracking from the moment you start pedaling.

In summary, designing a bike computer involves balancing trade-offs between cost, power efficiency, and performance. It's a delightful intersection of hardware, software, and practical considerations. So, whether you're cruising along scenic trails or pushing your limits on the road, your trusty bike computer has your back!

Engine Control Units (ECUs) and Their Demands

An engine control unit is like the brain of your car's powertrain. It orchestrates a symphony of processes to ensure your engine runs smoothly, efficiently, and reliably. Buckle up as we explore the fascinating details:

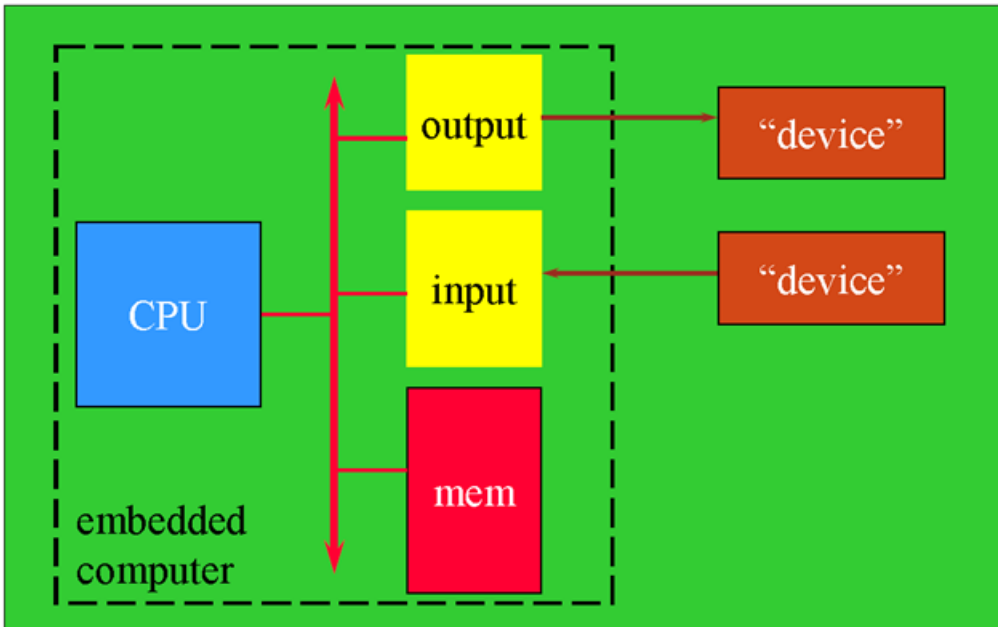
1. **Functional Requirements:**
   - Unlike our friendly bike computer, an ECU faces monumental responsibilities. It's responsible for critical tasks like spark timing and fuel injection.
   - Why are these crucial? Well, every spark ignites the air-fuel mixture in the cylinders, and precise fuel injection ensures optimal combustion. Mess up either, and your engine might grumble and hiccup like a disgruntled dragon.
2. **Real-Time Calculations:**
   - Here's the high-pressure part: ECUs must perform real-time calculations. Each spark and fuel injection event has a tight deadline.
   - Imagine a conductor leading an orchestra—timing matters. If the calculations aren't done by the beat, your engine stumbles, misfires, and generally behaves like it stayed up too late at the automotive party.
3. **Reliability and Harsh Environments:**
   - Cars face all sorts of weather—sweltering summers, icy winters, and monsoon madness. ECUs need to be tough cookies.
   - They operate in a harsh environment with temperature extremes, vibrations, and electrical noise. Reliability is non-negotiable; nobody wants their ECU throwing a tantrum during rush hour traffic.
4. **Cost Constraints:**
   - Car manufacturers want ECUs that don't break the bank. After all, they've got cupholders and panoramic sunroofs to worry about too.
   - So, balancing performance with cost efficiency is like tightrope walking while juggling wrenches.
5. **Space Limitations:**
   - Pop the hood, and you'll see it's like Tetris under there. Space is at a premium.
   - The ECU needs to fit snugly, like a well-tailored suit, amidst all the hoses, belts, and mysterious car parts.
6. **Sensor and Actuator Management:**
   - ECUs play matchmaker between sensors (like oxygen sensors, crankshaft position sensors, and coolant temperature sensors) and actuators (fuel injectors, ignition coils, and more).

- They collect data from sensors, make decisions, and tell actuators what to do. It's like a car-based matchmaking app.

7. **Microcontroller Selection:**
   - These microcontrollers handle complex algorithms, manage sensor data, and execute precise instructions. Think of them as the car whisperers.
   - Use high performance microcontroller e.g. 32-bit, 3 MB flash memory, 150 -300 MHz

In summary, designing an ECU is like crafting a Swiss watch: precision, resilience, and a dash of wizardry. So next time you turn the key (or press that fancy start button), remember that your ECU is working tirelessly behind the scenes, ensuring your ride is smooth, efficient, and drama-free.

Embedding a computer



Options for Building Embedded Systems

	Implementation	Design Cost	Unit Cost	Upgrades & Bug Fixes	Size	Weight	Power	System Speed
Dedicated Hardware	Discrete Logic	low	mid	hard	large	high	?	very fast
	ASIC	high ($500K/ mask set)	very low	hard	tiny - 1 die	very low	low	extremely fast
	Programmable logic – FPGA, PLD	low	mid	easy	small	low	medium to high	very fast
Software Running on Generic Hardware	Microprocessor + memory + peripherals	low to mid	mid	easy	small to med.	low to moderate	medium	moderate
	Microcontroller (int. memory & peripherals)	low	mid to low	easy	small	low	medium	slow to moderate
	Embedded PC	low	high	easy	medium	moderate to high	medium to high	fast

1) Microprocessors can be very efficient:

- Use same logic to perform many different functions.
- Create families of products.
- Create upgradable systems.

2) **Alternatives**:

- Custom System on Chip (SoC) implemented with ASICs, field-programmable gate arrays (FPGAs), etc. May or may not include microprocessor
- "Platform" FPGA –implement one or more microprocessor hard/soft cores, with embedded memory and programmable logic

- **Microcontroller:** includes I/O devices, on-chip memory.
- **Digital signal processor (DSP):**

  microprocessor optimized for digital signal processing.
- **Application-Specific Processor (ASP):**

  instruction set & architecture tailored to application (graphics, network, etc.)
- **Soft core:**

  microcontroller or CPU model to be synthesized into a system on chip (SoC)
- **Hard core:**

  microcontroller or CPU implemented as part of a SoC, "platform FPGAs"

Chapter 2 ARM7 Architecture

Main Points

- ARM IP Core
- Arm processor families
- Comparison of ARM Cortex A, M, R CPU's
- Arm processors vs. Arm architectures
- Features of ARM Processor {ARM7}
- Little Endian & Big Endian
- ARM7 data types
- ARM7 current program status register (CPSR)
- Modes of ARM7
- ARM7 programming model
- ARM7 Pipelining
- Memory with ARM 7
- Cache memory

ARM IP Core

What is IP Core?

IP is Intellectual Property owned by the company providing it to other companies in their systems. Core can be Microprocessor, DSP, Ethernet, Peripheral Cores etc. IC design engineers can use this IP Core in their system.

Types of IP Core:

1. Hard IP Core
2. Soft IP Core

➢ Hard IP Core
- It is a package chip with plastic case removed.
- Electrical properties are fixed.
- Size, Function and Shape are Fixed.
- Silicon process is also fixed.

➢ Soft IP Core
- It is a soft copy of source file for given design.
- Size and Shape is scalable with respect to technology.
- Functions are fixed, Source codes can not be modified.
- So, Source file is encrypted or delivered in unreadable form.

ARM Reference Methodologies

- Many companies were using IP Core of ARM for there applications.
- ARM works with most leading EDA (Electronic Design Automation) companies (Cadence, Magma, Synopsis, Mentor Graphics etc).
- EDA companies turns Soft IPs into Hard IPs using different technologies of fabrication and serve the needs of industries.
- ARM sells it's IPs to many industries {Apple, NVDIA, Qualcomm, Sony, INTEL, Samsung Motorola etc.).

Arm processor families

- **Cortex-A series (Application)**High performance processors capable of full Operating System (OS) support

Applications include smartphones, digital TV, smart books

- **Cortex-R series (Real-time)**

High performance and reliability for real-time applications;

Applications include automotive braking system, powertrains

- **Cortex-M series (Microcontroller)**

Cost-sensitive solutions for deterministic microcontroller applications

Applications include microcontrollers, smart sensors

- **SecurCore series** High security applications
- Earlier classic processors including Arm7, Arm9, Arm11 families

Comparison of ARM Cortex A, M, R CPU's

Parameters	Cortex A	Cortex R	Cortex M
Performance	❖ Highest	❖ Very Good	❖ Medium
Response Time	❖ Very Good	❖ Best	❖ Medium
Power Consumption	❖ 80µW/MHz	❖ 120µW/MHz	❖ 8µW/MHz
Processor	❖ Application Based	❖ RTOS based	❖ Embedded System based
Pipeline	❖ Long Pipeline	❖ Medium Pipeline	❖ Short Pipeline
Clock	❖ High	❖ High	❖ Less relatively
Memory	❖ Cache Memory with more size.	❖ Cache Memory + Tightly Coupled Memory	❖ Cache Memory with less size.
ISA	❖ ARM	❖ ARM	❖ Thumb
FPU	❖ Yes	❖ Yes	❖ Optional

Arm processors vs. Arm architectures

Arm architecture

- Describes the details of instruction set, programmer's model, exception model, and memory map
- Documented in the Architecture Reference Manual

Arm processor

- Developed using one of the Arm architectures
- More implementation details, such as timing information
- Documented in processor's Technical Reference Manual

Arch. Name	Processor / Core	Date	Notes
ARMv1	Arm1	1985	First Arm prototype
ARMv2	Arm2	1986	Coprocessor introduction 3-level pipeline, 24-bit addressing (16 registers of 32 bits, thus 64 MiB addressable)
ARMv2a	Arm3 Arm250	1989	Cache added
ARMv3	Arm6, Arm7	1991–1994	MMU added 32-bit addressing T - Thumb state M - long multiply support Interface FPU, 4K cache
ARMv4	StrongArm Arm7TDMI Arm8 Arm8TDMI	1993 →	T - Thumb
ARMv5	Arm7EJ, Arm9E, Arm10E, Arm1020, XScale, FA626TE, Feroceon, PJ1/Mohawk	1998 →	E - Enhanced DSP instructions, saturated arithmetic J - support for Jazelle, java byte code execution acceleration
ARMv6	Arm11	2002 →	Include T, E and J extensions MMU Multiprocessor facilities Support for 64-bit buses Mixed-endian support
ARMv6-M	Cortex-M M0, M0+, M1	2007	**Introduction of low-end product to Cortex-M family**; lowest power consumption
ARMv7-A	Cortex-A A5, A7, A8, A9 MPCore, -A5, A7, A12, A15	2005 →	**Start of Cortex-A family** Multicore Arm & Thumb instructions Virtual address support Thumb-2
ARMv7-R	Cortex-R R4, R5, R7	2011	**Start of Cortex-R family** Arm & Thumb instructions Physical address only
ARMv7-M	Cortex-M M3	2004	**Start of Cortex-M family** Thumb instruction set only
ARMv7E-M	Cortex-M M4, M7	2010 →	Enhanced DSP instructions, saturated arithmetic

Arch. Name	Processor / Core	Date	Notes
ARMv8-A **ARMv8.0-A**	Cortex-A A32, A35 A53, A57, A72, A73 Apple	2011–2019	32-bit architecture 64-bit architecture Multicore
ARMv8-R	Cortex-R R52, R53, R82	2016 →	AArch64
ARMv8-M **Baseline**	Cortex-M M23	2016	TrustZone on Cortex-M
ARMv8-M **Mainline**	Cortex-M M33, M35P	2016 →	TrustZone on Cortex-M FPU Coprocessor
ARMv8.1-A	Qualcomm/Cavium	2017 →	AArch64
ARMv8.1-M	Cortex-M M55	2020	New capabilities for AI (Artificial Intelligence) Helium instructions
ARMv8.2-A	Cortex-A A55, A65, A75, A76, A77, A78 Qualcomm/Samsung/ Fujitsu/HiSilicon/Apple	2017 →	AArch64
ARMv8.2-A	Cortex-X1	2020	Processor customization
ARMv8.3-A	Apple	2017 →	AArch64
ARMv8.4-A	Apple	2019 →	AArch64

Features of ARM Processor {ARM7}

- Originally developed by Arcon Computers in mid 1980s then renamed to ARM (Advanced RISC Machine).
- 'ARM Family' has verities of 32bits controllers used in many commercial applications, Embedded Systems, Smart Phones (Cortex A), Printers (Cortex R), Medical Instruments (Cortex M), etc.
- ARM processor accounts more then 75% of 32bits of processors in market share of industry in 2009.
- **Features of ARM7**
  - ARM7 has 32 bits of MCU & ALU.
  - ARM7 has 32 bits of Data bus with aligned memory space. (Means with each machine cycle it will execute 32bits via data bus. Here aligned addressing is done.)
  - ARM7 has all the instructions with 32 bits.
  - ARM7 has 32 bits of Address Bus. (So we can interface 4GB memory with it directly.)
  - ARM7 follows Von Neuman Model. (So 4GB memory will be common for code and data.) with latest versions they have switched to Harvard Architecture.
  - ARM7 has three stage Pipelining
    - Fetch
    - Decode
    - Execute
  - ARM7 has 37 registers of 32 bits. At time 16 registers available (RO-R15).
  - ARM7 has LOAD - STORE Architecture. (Means for LOAD and STORE operations we have to use separate instructions.)
  - ARM7 has 7 operating Modes.
  - ARM7 has 7 interrupts/Exceptions. ARM7 has 7 Addressing Modes.
  - ARM7 data formats (8bits Byte, 16bits - Half word, 32bits - Word)
  - ARM7TDMI
    - T-Thumb Instructions
    - D-Hardware Debugging
    - M-Long Multiply instruction
    - I-ICE Microcells for Breakpoints and watch points in program.

Little Endian & Big Endian

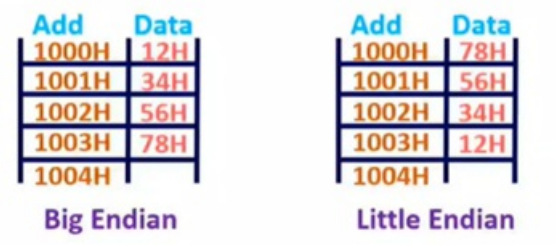
There are two ways of ordering the data while storing it into the Memory.

- Big Endian
- Little Endian

Big Endian: Most Significant Byte stored 1st in the memory and Last significant Byte stored at last in the memory.

Little Endian: Last Significant Byte stored 1st in the memory and Most significant Byte stored at last in the memory.

➢ Suppose we want to store 12345678H data starting at address 1000H.



➢ As we also operate with stack memory in CPU, we can remember this as per:
  - In Big Endian, Lower Address holds higher Byte of data and Higher Address holds lower Byte of data.
  - In Little Endian, Lower Address holds Lower Byte of data and Higher Address holds Higher Byte of data.

➢ Big Endian examples: Motorola 68xx, IBM Mainframe, TCP/IP etc.

➢ Little Endian examples: Intel x86, AMD, PIC etc.

➢ Advantages of Big Endian:

1. Easier to determine sign
2. Easier to compare two number
3. Easier to divide two number
4. Easier to Print

➢ Advantages of Little Endian:

1. Easier for Multiplication & Addition of multi-precision numbers.

ARM7 data types

- Unsigned numbers are extended by inserting O's ahead number.
- Positive numbers are extended by inserting O's ahead number and negative numbers are extended by inserting 1's ahead numbers.

Example:

If Data = +7 = 0000 0111b then

it will be extended as 0000 0000 0000 0000 0000 0000 0000 0111b

Example:

If Data71111 1001b then

it will be extended as 1111 1111 1111 1111 1111 1111 1111 1001b

- Here in binary for signed number, MSB shows polarity of data. (+Ve or-Ve Data)
- In most of the computers negative number is accounted as per 2's compliment of given number in signed number representation.

NOTE: In ARM, it supports load & store architecture, so in single instruction we can load data from memory into register or we can store data of register into memory.

Data Types	Size	Storing Condition	Extended After Loading
Byte	8 Bits	Can be stored at any location	❖ After loading Data from memory location, Data may have size of 8bits, 16 bits or 32 bits. ❖ But that data must be extended into 32 bits, as it must be copied into register and size of register is of 32 bits.
Half Word	16 Bits	16 Bits must be Aligned, Must Begin at even address {0, 2, 4 ...}	
Word	32 Bits	32 Bits must be Aligned, Must Begin at multiple of 4 {0, 4, 8 ...}	

ARM7 current program status register (CPSR)

The following figure shows that this register is composed of 32 bits.

The least significant five bits determines the ARM7 modes.

The following table illustrates these modes

Processor mode [1]		Mode encoding [2]	Privilege	Description
User	usr	10000	Unprivileged	Suitable for most application code.
FIQ	fiq	10001	Privileged	Entered as a result of a fast interrupt. [3]
IRQ	irq	10010	Privileged	Entered as a result of a normal interrupt. [3]
Supervisor	svc	10011	Privileged	Suitable for running most kernel code. Entered on Reset, and on execution of a Supervisor Call (SVC) instruction.
Monitor [4]	mon	10110	Privileged	A Secure mode that enables change between Secure and Non-secure states, and can also be used to handle any of FIQs, IRQs and external aborts. [3] Entered on execution of a Secure Monitor Call (SMC) instruction.
Abort	abt	10111	Privileged	Entered as a result of a Data Abort exception or Prefetch Abort exception. [3]
Undefined	und	11011	Privileged	Entered as a result of an instruction-related error.
System	sys	11111	Privileged	Suitable for application code that requires privileged access.

- **T-Thumb State**

  1 = Thumb State

  0 = ARM State

  In Thumb State, we have 16bits of instruction, it useful for increasing code density.

Parameters	ARM State	Thumb state
CPSR T bit	❖ T = 0	❖ CPSR T = 1
Instruction Size	❖ 32 bits	❖ 16 bits
Core Instructions	❖ 68	❖ 30
Conditional Execution	❖ Most	❖ Only Branch Instructions
Data Processing Instructions	❖ Access to Barrel Shifter and ALU	❖ Separate Barrel Shifter and ALU Instructions.
CPSR Accessibility	❖ Full Access in System Mode	❖ No direct Access
Register Usage	❖ 15 General Purpose Registers + PC : Program Counter	❖ 8 General Purpose Registers + 7 High Registers + PC : Program Counter
Code Density	❖ Low Code density as all the instructions size is 32 bits.	❖ High Code density as all the instructions size is 16 bits
System Use	❖ Full use of System in ARM State	❖ Partial use of system in Thumb State
Applications	❖ High Profile applications.	❖ Low Profile Applications.
Memory Requirement	❖ More memory required as size of instructions is 32 bits.	❖ Less memory required as size of instructions is 16 bits.

- F-Fast Interrupt Mask

  1 = Fast Interrupts Masked

  0 = Fast Interrupts Unmasked

  Fast Interrupts is given on nFIQ Pin.

- I - Interrupt Mask

  1 = Interrupts Masked

  0 = Interrupts Unmasked

Normal Interrupts is given on nIRQ Pin.

- V- Overflow Flag

  1 = Signed Overflow

  0 = No Signed Overflow

- C-Carry Flag

  1 = Carry After MSB

  0 = No Carry After MSB

- Z - Zero Flag

  1 = Result is Zero

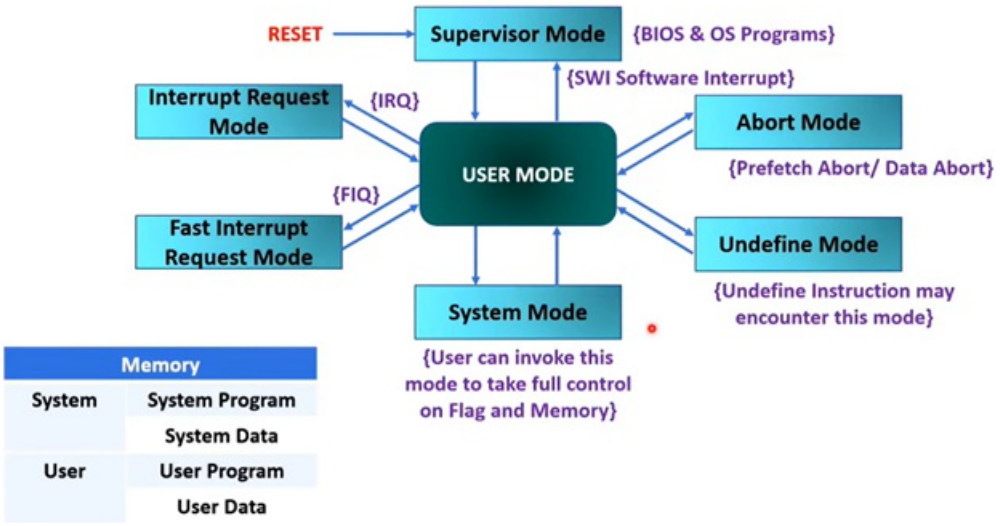
  0 = Result is not Zero

- N-Negative Flag

  1 = Result is Negative

  0 = Result is not Negative

Modes of ARM7

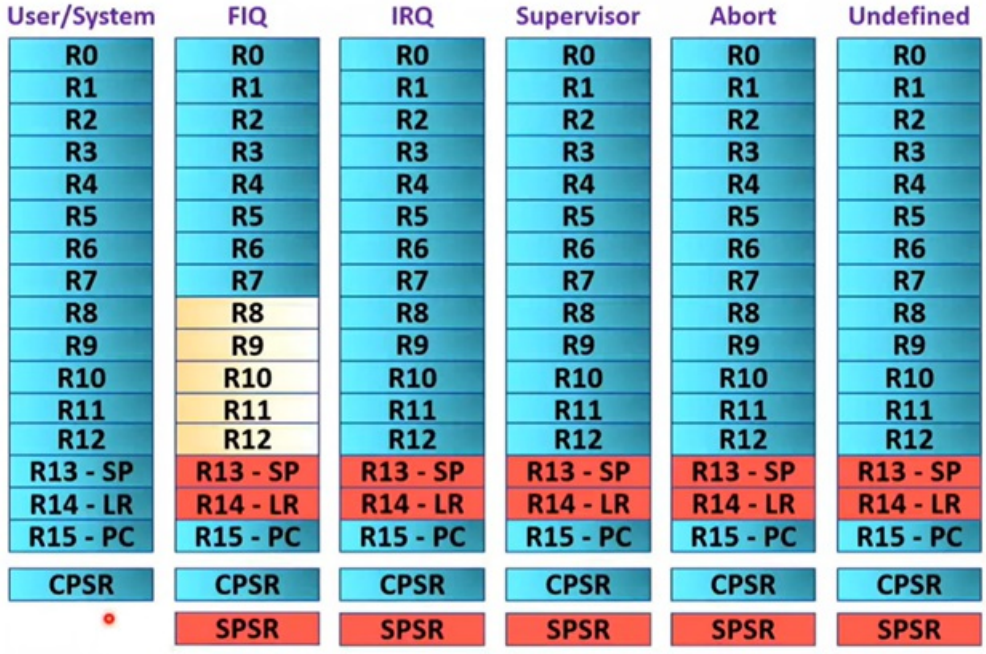


-

-

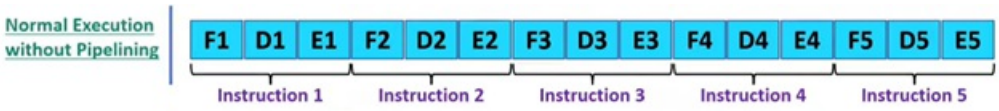
- **User Mode**
  - This is normal mode in which all the user programs are executed.
  - It is the only non privileged mode.
  - It has limited access to Memory, IO and Flags.
  - All other modes are entered through interrupt.
- **Fast Interrupt Mode**
  - This mode is enabled when high priority interrupt comes on nFIQ Pin.
  - This interrupts should be served with minimum delay.
  - By default Nested interrupts are enabled in this Mode.
- **Interrupt Mode**
  - This mode is enabled when normal interrupt comes on nIRQ Pin.
  - This interrupts will be served with some delay.
  - Nested interrupts are allowed in this Mode.
- **Supervisor Mode**
  - This mode is enabled when we reset the system.
  - It is used to execute BIOS program.
  - This mode can be invoked by programmer with SWI {Software interrupt}.
- **Abort Mode**
  - This mode is entered when unsuccessful attempt is made to access memory.
  - Due to protection mechanism some memory locations are not accessible.
- **Undefined Mode**
  - This mode is entered when undefined instructions attempted.
  - This generally happens when coprocessor instruction encountered but coprocessor is not available.

ARM7 programming model



- All the registers are orthogonal in ARM.
- Orthogonal means any instruction is Applied to RO is also applicable to any register.
- User/System Mode is having R0-R15 and CPSR register. In total 17 registers. Main Program is there in user mode only.
- Other Five modes is having R0-R15, CPSR & SPSR registers, Other modes are executed by some shorts of interrupt.
- So, other modes are served as ISR.
- PC - Program Counter: It holds the address of next instruction.
- LR - Link Register: It holds the data of PC during other modes execution & Branch execution {CALL}.
- CPSR - Current Program Status Register: It holds the status and control of program.
- SPSR - Saved Program Status Register: It holds the CPSR of User mode during other modes execution.
- SP - Stack Pointer: It holds the address of Top of stack memory. SP is separately provided to all the modes.
- FIQ Register - R8 to R12: This are dedicate registers to provide fast service to interrupt.
- So in total ARM7 have 17 User/System, 5 SP, 5 LR, 5 SPSR & 5 FIQ = 37 registers of 32 bits.

ARM7 Pipelining



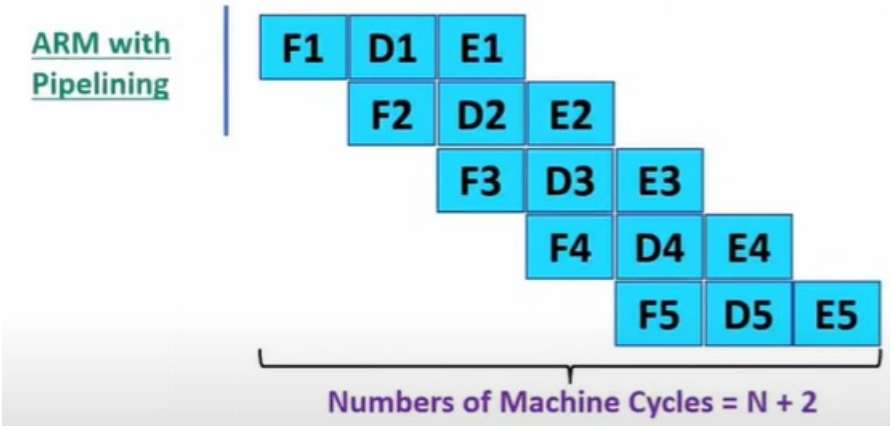
F stands for fetch

D stands for decode

E stands for Execute

Now,

- Fetch, Decode & Execute has equal size of Machine Cycle, Why?
- Fetch: All the instruction of ARM7 has size of 32 bits. So, it will take only one machine cycle to fetch it as ARM7 has 32 bits data bus with aligned locations.
- Decode: After fetch hardwire decode circuit will decode instructions in one machine cycle only.
- Execute: Execution is done by ALU & MAC with respect to registers only. So execution will take one machine cycle only.
- ARM7 has LOAD and STORE architecture. So, for loading the data from memory and storing data on to memory, there are separate instructions.



- As ARM7 support 3 stage pipeline, PC always points 2 instructions ahead of the one being executed now.

  Advantage: Fast Program Execution.

  Disadvantage: Pipeline Fails in Branch Instruction execution.

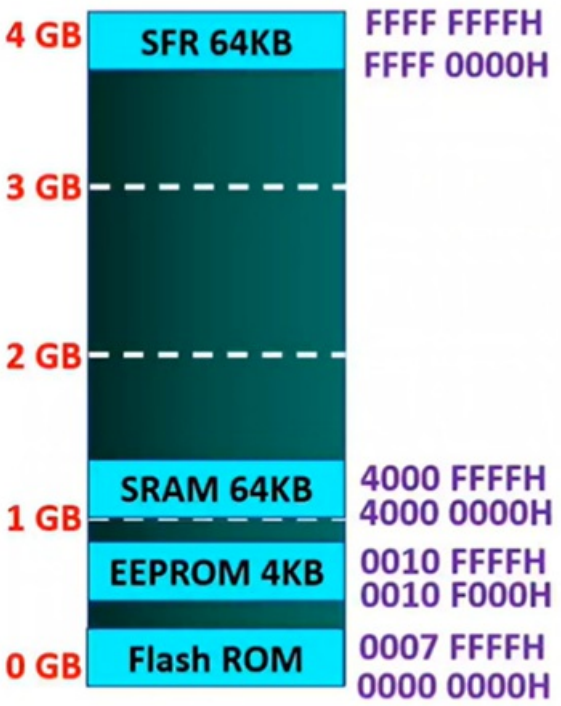
- ARM Pipelining

➢ ARM7 uses 3 stage pipeline: Fetch, Decode & Execute.

➢ ARM9 uses 5 stage pipeline: Fetch, Decode, Execute, Memory write & Register write.

➢ ARM10 uses 6 stage pipeline: Fetch, Issue, Decode, Execute, Memory write & Register write.

Memory with ARM 7



- Memory Allocation in ARM
  - ➢ The ARM has 4GB of Memory space.
  - ➢ This memory space has address from 0000 0000H to FFFF FFFFH.
  - ➢ This 4GB memory space can be divided into five sections:

    1. On Chip Peripherals and IO Registers:

       This area is dedicated to General purpose 10 GPIO and Special Function Registers - SFR of peripherals such as Timers, Serial Communication, ADC etc. Memory Mapped IO is used here. It can vary chip to chip of ARM7.

    2. On Chip Data SRAM:

       It ranges from few KB to serval hundreds of KB. It is used by data variables and STACK. It can vary chip to chip of ARM7. It's locations are well managed by C complier.

    3. On Chip EEPROM:

       It ranges from 1KB to serval hundreds of KB. It is used store some program codes, Most often used for critical program. It can vary chip to chip, it may not be available.

    4. On Chip Flash ROM:

       It ranges from few KB to serval hundreds of KB.

       It is used store program Space.

       It can also be used to store some fixed data like ASCII & Look up table.

       It is there under control of PC of ARM.

       It can vary chip to chip.

    5. OFF Chip DRAM:

It ranges from few MB to serval hundreds of MB.

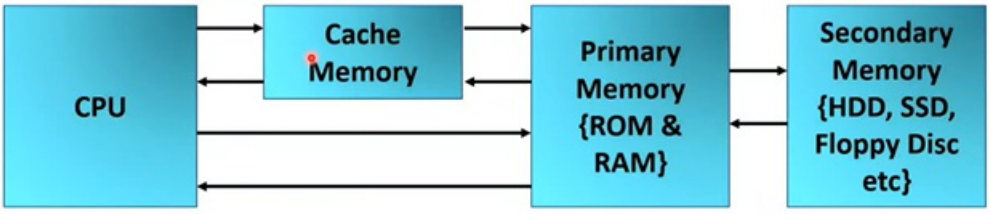
It can be implemented by external interfacing with ARM.

User can interface OFF Chip DRAM based on the need of application.

Cache memory

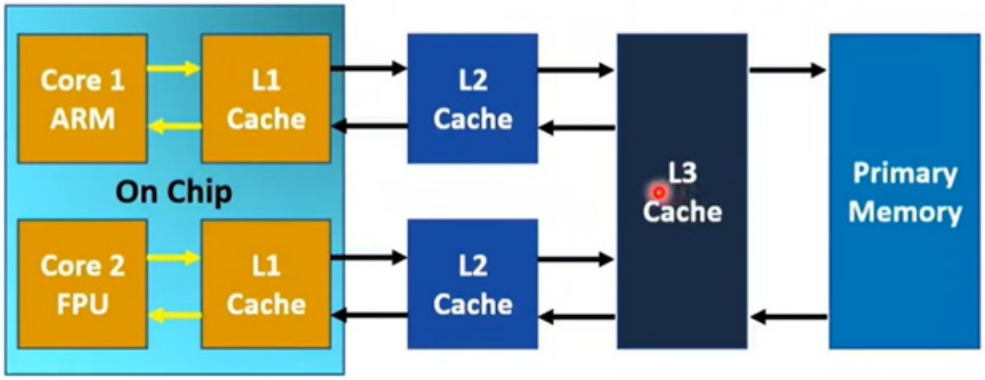
What is Cache Memory?

- It is SRAM.
- Cache Memory holds recently & frequently accessed files.
- Cache Memory may be there on chip or off chip.



Basics of Cache Memory

- Cache memory is small in size and it has fast data access.
- The part of data and program are transferred from Primary or secondary memory to cache memory by OS.
- If data from Primary Memory is mapped with the address inside cache memory then it is referred as 'Cache Hit' and If data from Primary Memory is not mapped with address inside cache then it referred as 'Cache Miss'.
- If data is not available in cache memory then by different memory mapping scheme, data will be mapped to cache memory.



Levels of Cache Memory

- Level L1 Cache or Primary Cache:
  - Small Size {2KB to 64KB}
  - It is Embedded with CPU {SOC}
  - It holds very important data like registers {Accumulator, PC, SP, etc.}
  - It is operates at same speed of CPU.
  - It is Nearest to CPU.
- Level L2 Cache or Secondary Cache:
  - Medium Size (256KB to 512KB}
  - It is outside CPU, in some case it may be inside chip

- It holds recently access files
- {Code & data}
- It is slower then L1.
- If data is not available in L1 then L1 will take it from L2.

➢ Level L3 Cache or Main Memory:

- Large Size (1MB to 8MB)
- In multi core system, each core have personal L1 & L2 but it will have common L3.
- It is having double speed of normal RAM.

Flush of Cache Memory

➢ When CPU is accessing data from cache and if it is available with cache memory then operations will be super fast.

➢ But if Outdated (No longer in use of core) data is there in cache then it has to get flush and there must be currently accessed data on cache memory for faster execution of programs.

➢ Operating system OS updates currently accessed files into cache memory for faster execution of programs.

➢ In order to execute program fast, OS will remove outdated data from cache is referred Flush of cache memory.

Clear Cache Memory

➢ If data is overloaded with cache memory, then program execution will be slower.

➢ Important programs may not work as per the expectations of programmer.

➢ By clearing cache memory, we can make execution fast.

➢ Clear cache decision is taken by programmer not by OS.

Advantages of Cache Memory

➢ Less Access time.

➢ Program execution is fast with Cache.

➢ It stores the data for temporary usage.

Disadvantages of Cache Memory

➢ Limited size.

➢ Costly

Chapter 3 ARM assembly programming

Main points

- Assembly language vs. C language
- ARM7 development tools
- Data formats for ARM
- ARM assembler directives
- Structure of ARM assembly instructions
- Types of ARM assembly instructions
- Addressing Modes of ARM7
- Data movement instructions
- Data processing instructions
- Program flow instructions
- Special Instructions of ARM7

Assembly language vs. C language

Assembly Language:

- **What Is It?**:

  Assembly language is a low-level programming language that acts as a bridge between human-readable code and machine code. Unlike high-level languages (such as Python or Java), assembly language provides a direct interface with the hardware. It's essentially a more human-friendly representation of the binary instructions that the computer's processor understands.

- **Control Over Hardware**:

  One of the most intriguing aspects of assembly language is the fine-grained control it offers over the microprocessor. Programmers get to manipulate registers, memory, and other hardware resources directly. It's like having a backstage pass to the inner workings of the computer!

- **Efficiency**:

  When performance truly matters—think system programming, real-time applications, or embedded systems—assembly code shines. Why? Because it's tailored to specific tasks. You can optimize it down to the last clock cycle, which is essential in scenarios where every ounce of efficiency counts.

- **Challenges**:

  Now, here's where things get interesting (and sometimes hair-pulling). Writing assembly can be complex and time-consuming. You're dealing with raw instructions, memory addresses, and low-level abstractions. Debugging? Oh, that can be an adventure too, especially since you don't have the luxury of high-level debugging tools.

- **Historical Context**:

  Picture this: Back in the early days of computing, assembly was the only game in town. Memory was scarce, and programmers needed tight control over its use. Assembly provided that control. But as technology evolved, higher-level languages like C came along. They offered a balance between control and productivity. Suddenly, you didn't have to count every byte manually.

1. **C Programming**:
   - **High-Level Abstraction**:

     C is a high-level language, which means it provides a more human-friendly way to express algorithms compared to assembly language. It's portable, widely used, and—bonus—it's easier to learn than diving straight into raw assembly code.

   - **Portability**:

     Imagine writing code that can pack its bags and travel across different platforms without much trouble.

   - **Compiler Magic**:

Modern C compilers act like masterful code interpreters, taking your high-level C code and translating it into optimized assembly instructions. This process, while advantageous, can create challenges for debugging, as you'll need to factor in the compiler's transformations to understand the code's behavior.

- **Close to the Hardware**:

  C sits close to the hardware, offering a balance between abstraction and direct access. While it simplifies some hardware complexities, it empowers you to write low-level code when needed. This is evident in device drivers, which often combine C with assembly language for optimal control.

-

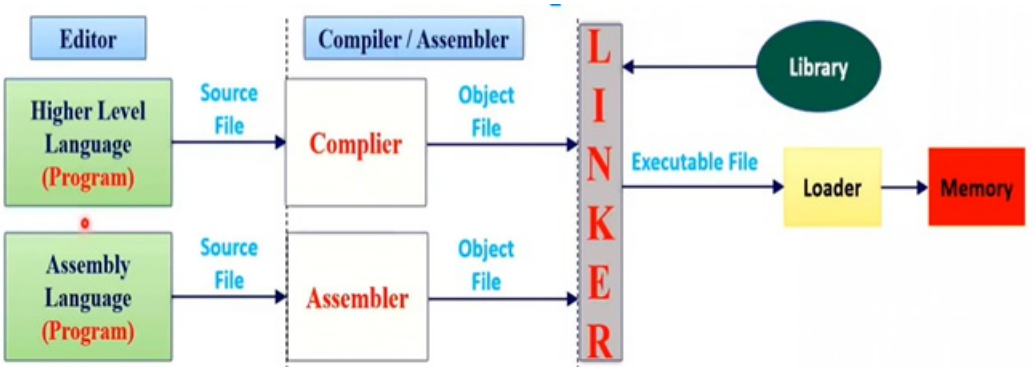
- **Community and Libraries**: C has a rich ecosystem of libraries and a large community.

2. **Why Learn Both?**:
   - **Enhanced Debugging and Optimization:** Understanding assembly provides insights into how compilers optimize your C code, making debugging more effective and allowing you to write more efficient programs.
   - **Unlocking Low-Level Control:** For critical sections or specialized tasks demanding maximum control, assembly knowledge can be a game-changer. It allows you to work directly with the machine's instructions, like peering behind the curtain of how your code operates.
   - **Bridging the Gap:** Assembly acts as a bridge between your C code and the raw instructions the computer understands. By understanding it, you gain a deeper appreciation for how high-level languages work and interact with the machine.

The following table summarizes the differences between assembly language and C language

Assembly Language	C Language
❖ Unstructured Language {No Formal Structure}	❖ Structured Language {Well Define Structure}
❖ Differs with respect to Processors & Controllers	❖ Core Idea remains same with respect to Processors & Controllers
❖ Low Level Interface	❖ High Level Interface. It can use Assembly as well.
❖ Architecture specific – Based on instruction set	❖ General Purpose
❖ Assembler converts it into Machine Codes.	❖ Compiler converts into Machine Codes.
❖ Program Length is Small	❖ Program Length is longer then Assembly.
❖ Program Execution is Faster	❖ Program Execution is Slower.
❖ Development of Program is bit complex.	❖ Development of Program is relatively easier.
❖ Less then 10% of programmers use Assembly.	❖ More then 90% of programmers use C.
❖ Less Flexible	❖ More Flexible
❖ Slow Development	❖ Fast Development
❖ Performance based programming	❖ User Friendly Programming

ARM7 development tools



- In Editor we write programs for ARM.
- Programs may be written in Assembly Language or Higher Level Language {C Language).
- By writing program we generate source file.
- Assembler It is used to convert Assembly language into machine code or object file. It also shows errors if any syntax error is there in program.
- Complier: It is used to convert Higher Level language into machine code or object file. It also shows errors if any syntax error is there in program. It also gives warnings if it is there with programs. {Kail}
- Linker: It is linking all the object files of compiler and assembler with the use of library.
- It will generate executable files.
- Loader: It is used to load executable files into the memory of ARM.
- Once program is loaded into memory, ARM can execute it as per the requirement of USER.

Data formats for ARM

- The Numbers can be represented in Hex, Binary, Decimal, Numbers in Any Base & ASCII Formats in ARM.
- Hex Numbers
  - To represent Hex numbers in an ARM assembler we put 0x (or 0X) in front of the number:
    - MOV R1, #0x99
    - MOV R2, #0x75
    - ADD R2, R1, R2
- Decimal Numbers
  - To indicate decimal numbers in some ARM assemblers such as Keil we simply use the decimal (e.g., 12) and nothing before or after it. Here are some examples of how to use it:
    - MOV R1, #99
    - MOV R2, #75 ADD R2, R1, R2
- Binary Numbers
  - To represent binary numbers in an ARM assembler we put 2_ in front of the number. It is as follows:
    - MOV R1, #2_10011001
    - MOV R2, #2_01110101 ADD R2, R1, R2
- Numbers in any base
  - To represent numbers with any base in an ARM assembler we put N_ in front of the number, where N is base. It is as follows:
    - MOV R1, #8_56
    - MOV R2, #5_34
    - ADD R2, R1, R2
- ASCII
  - To represent ASCII data in an ARM assembler we use single quotes as follows:
    - LDR R1, #'26 ;ASCII of '2' is 32H
- To represent string we use double quote.

ARM assembler directives

- Assembler Directives are used to given directions to assembler. It is also referred as Pseudo Codes.
  - AREA

    The AREA directive tells the assembler to define a new section of memory.

    The memory can be code (instruction) or data and can have attributes such as READONLY, READWRITE, COMMON and ALIGN.

    Format: AREA Name, attribute, attribute, attribute, ...

    - AREA Prog1, CODE, READONLY
    - AREA Variables, DATA, READWRITE, ALIGN
    - AREA Constants, DATA, READONLY, COMMON

    COMMON is an attribute used to define common data shared by servals programs.

    ALIGN is an attribute used to Align the stored data. We don't use it with code, as all the instructions are of 32bits only.

  - ENTRY and END

    ENTRY directive declares entry points of Program

    END directive is used to Terminate the Program. Anything after END directive will be ignored by assembler.

```
AREA Prog1, CODE, READONLY
    ENTRY
        MOV R1,#0x25H
        MOV R2,#0x55H
        ADD R3, R1, R2
    END
```

  - EQU

    EQU directive is used to define content value or fixed address.

    Format: Name EQU 0x25

  - RN

    RN directive is used to define name of register.

    Format: Name RN R1

  - LDR

    LDR directive is used to load 32bits of data into registers.

    To load data greater then 8 bits we can use LDR directive.

    Format: LDR R1,=0x23232323

LDR is also instruction available in ARM.

Format of LDR Instruction: LDR R1, [R5] or LDR R1, Label

- DCB, DCW, DCWU, DCD, DCDU
  - DCB Byte. Define Constant Byte: We can Define one or more
  - DCW-Define Constant Word: We can Define one or more 16 bits of data.
  - DCD-Define Constant Double Word: We can Define one or more 32 bits of data.
  - DCWU-Define Constant Word: We can Define one or more 16 bits of data. Data is not Aligned.
  - DCDU-Define Constant Double Word: We can Define one or more 32 bits of data. Data is not Aligned.
  - Format: Name DCW 0x2532, 0x2353
- SPACE

SPACE directive is used to allocate space to variables.

Format: Name SPACE N ; N numbers of Bytes to Name

- ADR

ADR directive is used to load address into register.

Format: ADR Rn, Label

So, this directive will load address of Label into register Rn.

- ALIGN

ALIGN directive is used to aligned the data.

Format: ALIGN 4; The next instruction is 4 Bytes Aligned.

Format: ALIGN 2; The next instruction is 2 Bytes Aligned.

Structure of ARM assembly instructions

All ARM instructions conform to the following structural layout:

label <space> opcode <space> operands <space> ; comment

Each field (label, opcode, operands and comment) must be separated by one or more <space> elements. A <space> element can be an actual space character (from your space bar) or a tab.
If there is no label, there must be a <space> element before the actual instruction (i.e. the opcode and any operands.)
The semicolon character indicates the start of a comment, which is terminated by the end of the line.
All sections are optional. Blank lines will be accepted by the assembler, making it easier to improve code clarity.

Types of ARM assembly instructions

There are three broad categories of instruction in the ARM assembly language. These are:

- **Data Processing**: this includes arithmetic and logical operations, comparison operations and register movement operations;
- **Data Movement:** these are instructions to load and store data from and to memory;
- **Control Flow:** these are software interrupts and branch instructions that alter the order of execution.

For ARM data processing instructions, operands (values) are always 32 bits wide. The operands are either held in registers or are specified as constants (called *literals*) in the instruction itself. The result of a data processing instruction is also a 32 bit datum and is stored in a register. Most data processing instructions will have three operands, two of which are inputs and one for the result.

Addressing Modes of ARM7

Addressing mode: The Various formats of specifying the operands in instructions are called addressing modes. (programmer view)

- Immediate Addressing Mode
- Register Addressing Mode
- Direct Addressing Mode
- Indirect Addressing Mode
- Register Relative Indirect Addressing Mode
- Base Indexed Indirect Addressing Mode
- Base with scaled Index Indirect Addressing Mode

Immediate Addressing Mode

In this Addressing mode, data (Only 1byte) specified in instruction.

Example:

MOV R5, #0x20 ; 20H will copied to R5

ADD R0, R1, #0x20 ; R0 ← R1 + 20H

Register Addressing Mode

In this Addressing mode, data is given by registers only.

All logical and Arithmetic instructions are based on this mode only with ARM.

Example:

MOV R5, R1 ; R1 will be copied into R5

ADD R0, R1, R2 ; R0 ←R1 + R2

Direct Addressing Mode

In this Addressing mode, Address of operand is given in the instruction.

This Address will be 12bits offset from PC given by assembler.

Example:

LDR R5, Variable ; R5 ← [Variable]

STR R5, Variable ; [Variable] ← R5

Indirect Addressing Mode

In this Addressing mode, Address of operand is given by register.

Example:

LDR R5, [R1] ; R5 - [R1]

STR R5, [R1] ; [R1] - R5

Register Relative Indirect Addressing Mode

Here address of the memory operand is given by a register + Numeric Value.

Example:

LDR R0, [R1, #0x04] ; R0 ← data from memory pointed by (R1+4).

Here R1 remains unchanged.

LDR R0, [R1, #0x04] ! ; First, R1 ← R1+4

Then, R0 ← data from memory pointed by (R1).

• This is called PRE-INDEX addressing.

LDR R0, [R1], #0x04 ; First, R0 ← [R1]

Then, R1 ← R1+4

• This is called Post-INDEX addressing.

Base Indexed Indirect Addressing Mode

Here address of the memory operand is given by a sum of two registers.

Where first register act as base and second register act as Index register

Example:

LDR R0, [R1, R2] ; R0 ← data from memory pointed by (R1+R2).

Here R1 remains unchanged.

LDR R0, [R1, R2]! ; First, R1 ← R1+R2

Then, R0 ← data from memory pointed by (R1).

- This is called PRE-INDEX addressing.

LDR RO, [R1], R2 ; First, R0 ← [R1]

Then, R1 ← R1+R2

- This is called Post-INDEX addressing.

Base with scaled Index Indirect Addressing Mode

Here address of the memory operand is given by a

sum of two registers, Where first register act as base and second register scaled index by shift left.

Example:

LDR R0, [R1, R2, LSL #2]

R0 ← data from memory pointed by (R1+R2 shifted left by 2 bits).

Here R1 remains unchanged.

Data movement instructions

1) Data transfer instructions

- Normal Mov instruction:

  MOV R0, R1

- Immediate data Mov instruction:

  MOV R0, #0x25

  Note: Only 8bits data can be moved immediately.

- Normal Mov instruction which affect the flag:

  MOVS R0, R1

  If R1=0, Then after MOVS RO, R1 value of RO = 0, but it will affect the zero flag Z = 1.

  If R1 is Negative, Then after MOVS RO, R1 value of RO = R1, but it will affect the Negative flag N = 1.

- Conditional Mov instruction: It performs move based on pervious instruction status.

  MOVCC RO, R1

  So, with previous instruction, if C = 0 then only R1 will be only into R0, else it will skip this instruction.

- Conditional Mov instruction which effect the flag: It performs move based on pervious instruction status, also it will effect the flag after execution.

  MOVCSS RO, R1

  CS-Carry Set, C = 1

  CC-Carry Clear, C = 0

  VS-Overflow Set, V = 1

  VC-Overflow Clear, V = 0

  MI-Minus, N = 1

  PL-Plus, N = 0

  EQ-Equal, Z = 1

  NE- Not Equal, Z = 0

  HI-1st Number is Higher

  HS-1st Number is Higher or Same

  LO-1st Number is Lower

  LS-1st Number is Lower or Same

  GT-1st Number is Greater than (Signed}

GE-1st Number is Greater or Equal (Signed}

LT-1st Number is Less Than {Signed}

LE-1st Number is Less or Equal {Signed}

- Mov Not instruction: It performs move with NOT Operation.
  MVN R0, R1
- With MVN instruction we can use conditions as well as status implementation.

  MVNVSS R0, R1

2) Load and store instructions

➢ **Load instructions**

- LDR-Load Register from given memory location {32 bits data}

  LDR RO, [R1]

  It will load RO with the data pointed at memory location by R1, it will load 32bits data.

- LDRB-Load Register from given memory location (8 bits data}

  LDRB RO, [R1]

  It will load RO with the data pointed at memory location by R1, it will load 8bits data.

- LDRH-Load Register from given memory location {16 bits data}

  LDRH RO, [R1]

  It will load RO with the data pointed at memory location by R1, it will load 16bits data.

- LDRSB-Load Register from given memory location (8 bits signed data}

  LDRSB RO, [R1]

  It will load RO with the data pointed at memory location by R1, it will load 8bits signed data.

- LDRSH Load Register from given memory location {16 bits signed data}

  LDRSH RO, [R1]

  It will load RO with the data pointed at memory location by R1, it will load 16bits signed data.

Note: If this instructions are written with T subscript then it will execute instruction with user privileged. It will stop access of system data

LDRT RO, [R1]

➢ **Store instructions**

- STR-Store Register on given memory location (32 bits data}

  STR RO, [R1]

  It will store RO at memory location pointed R1, it will Store 32bits data.

- Similarly,
  - STRB-Store Register on given memory location (8 bits data}

- STRH - Store Register on given memory location {16 bits data)
- STRSB-Store Register on given memory location (8 bits Signed data}
- STRSH Store Register on given memory location {16 bits Signed data)

Flag related instructions

- MRS - Mov into Register from Status 。

  MRS RO, CPSR

  It will move CPSR into RO register. Here, instead of RO, we can use any other register as well.

  MRS RO, SPSR

  It will move SPSR into RO register. Here, instead of RO, we can use any other register as well.

- MSR - Mov into Status from Register

  MSR CPSR, RO

  It will move RO into CPSR register. Here, instead of RO, we can use any other register as well.

- MSR SPSR, RO

  It will move RO into SPSR register. Here, instead of RO, we can use any other register as well.

Data processing instructions

1) Arithmetic instructions

- ADD-Normal Addition

  ADD RO, R1, R2

  It will add R1 & R2, result will be stored in RO. Flag will remain unchanged.

- ADDS-Normal Addition with flag update

  ADDS RO, R1, R2

  It will add R1 & R2, result will be stored in RO. Appending the instruction with "S" means that Flags will change with respect to R1 + R2.

- ADDEQS-Conditional Addition with flag update

  ADDEQS RO, R1, R2

  It will add R1 & R2 only if previous instruction has flag Z=1, result will be stored in RO. Flag will change with respect to R1 + R2 ("S" is appended). Following are the conditions that can be used in "conditional addition"

  CS-Carry Set, C = 1

  CC-Carry Clear, C = 0

  VS-Overflow Set, V = 1

  VC-Overflow Clear, V = 0

  MI-Minus, N = 1

  PL-Plus, N=0

  EQ-Equal, Z = 1

  NE-Not Equal, Z = 0

  HI-1st Number is Higher

  HS-1st Number is Higher or Same

  LO-1st Number is Lower

  LS-1st Number is Lower or Same

  GT-1st Number is Greater than {Signed}

  GE-1st Number is Greater or Equal (Signed}

  LT-1st Number is Less Than {Signed}

  LE-1st Number is Less or Equal {Signed}

- ADC-Addition with Carry

  ADC RO, R1, R2

It will add R1 & R2 with carry, result will be stored in RO. Flag will remain unchanged.

As with ADD instruction you can append "S" to affect the flags and also you can append conditions to perform conditional addition with carry.

- SUB/SBC- Subtraction

SUB RO, R1, R2

It will subtract R1-R2 in SUB, result will be stored in RO. Flag will remain unchanged.

SBC RO, R1, R2

It will subtract R1-R2- Carry in SBC, result will be stored in RO. Flag will remain unchanged.

As with ADD instruction you can append "S" to affect the flags and also you can append conditions to perform conditional subtraction.

- RSB/RSC- Reverse Subtraction

RSB RO, R1, R2

It will subtract R2-R1 in RSB, result will be stored in RO. Flag will remain unchanged.

RSC RO, R1, R2

It will subtract R2-R1 - Carry in RSC, result will be stored in RO. Flag will remain unchanged.

As with ADD instruction you can append "S" to affect the flags and also you can append conditions to perform conditional reverse subtraction.

Multiplication instructions

* ARM supports Multiply instructions and ARM also supports Multiply & Accumulate instructions.

- MUL-Normal Multiplication

MUL RO, R1, R2 ;RO←R1 x R2

It will Multiply R1 & R2, result will be stored in RO. Flag will remain unchanged. To get answer with flag, we should use MULS.

- MLA-Multiplication & Accumulation

MLA RO, R1, R2, R3 ;RO← R1 x R2 + R3

It will Multiply R1 & R2 also it will add R3 with it, result will be stored in RO. Flag will remain unchanged. To get answer with flag, we should use MLAS.

- MUL & MLA works with unsigned numbers only.
- Technically, 32bits x 32bits = 64bits, but as ARM supports 8bits, 16bits & 32bits data, programmer can use MUL & MLA for 8bits & 16bits data multiplication.
- UMULL-Unsigned Long Multiplication

UMULL RO, R1, R2, R3 ;(R1,R0)← R2 x R3

It will Multiply R2 & R3, result will be stored in (R1, RO). Flag will remain unchanged. To get answer with flag, we should use UMULLS.

So, Here in Long multiplication answer will of 64bits.

- SMULL-Signed Long Multiplication

  SMULL RO, R1, R2, R3 ;(R1,R0)← R2 x R3

  It will Multiply R2 & R3, result will be stored in (R1, RO). Flag will remain unchanged. To get answer with flag, we should use SMULLS.

  So , Here in Long multiplication answer will of 64bits.

- UMLAL - Unsigned Long Multiplication & Accumulation UMLAL RO, R1, R2, R3 ;(R1,R0) ←R2 x R3 +(R1,RO)

  It will Multiply R2 & R3 also it will add (R1, RO), result will be stored in (R1, RO). Flag will remain unchanged. To get answer with flag, we should use UMLALS.

  So, Here in Long multiplication answer will of 64bits.

- SMLAL-Signed Long Multiplication & Accumulation SMLAL RO, R1, R2, R3 ;(R1,R0)← R2 x R3 +(R1,RO)

  It will Multiply R2 & R3 also it will add (R1, RO), result will be stored in (R1, RO). Flag will remain unchanged. To get answer with flag, we should use SMLALS.

  So, Here in Long multiplication answer will of 64bits.

2) Logic instructions

- AND-Logical AND Operation

  AND RO, R1, R2 ;RO← R1 AND R2

  It will load RO with logical AND operation of R1 and R2.

  It will not update flag, to get result with updated flag use ANDS.

- ORR-Logical OR Operation

  ORR RO, R1, R2 ;RO ← R1 OR R2

  It will load RO with logical OR operation of R1 and R2.

  It will not update flag, to get result with updated flag use ORRS.

- EOR-Logical XOR Operation

  EOR RO, R1, R2 ;RO ← R1 XOR R2

  It will load RO with logical XOR operation of R1 and R2.

  It will not update flag, to get result with updated flag use EORS.

- BIC-AND with Compliment

  BIC RO, R1, R2 ;RO ← R1 AND $\overline{R2}$

  It will load RO with logical AND operation of R1 and $\overline{R2}$.

  It will not update flag, to get result with updated flag use BICS.

- CMP-Compare
  CMP RO, R1 ; RO - R1

  It performs RO-R1, but does not stores result of it.

  It is just affecting flag for comparison, here 'S' is not required to change the status.

- CMN-Compare with Negated Value
  CMN RO, R1 ; RO + R1

  It performs RO + R1, but does not stores result of it.

  It is just affecting flag for comparison, here 'S' is not required to change the status.

- TST-Test logic AND Operation
  TST RO, R1 ; RO AND R1

  It performs RO AND R1, but does not stores result of it.

  It is just affecting flag for comparison, here 'S' is not required to change the status.

- TEQ-Test logic XOR Operation TEQ RO, R1 ; RO XOR R

  It performs RO XOR R1, but does not stores result of it.

  It is just affecting flag for comparison, here 'S' is not required to change the status.

Program flow instructions

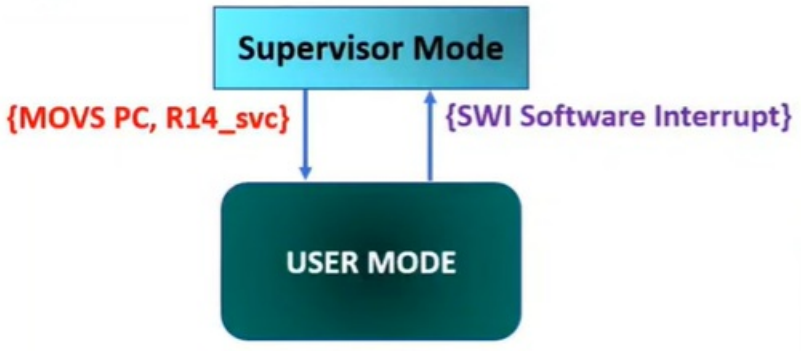
Conditional instructions of ARM7

Most Instructions of ARM is conditional instructions. That's why it kills majority of branching probability. There are 16 possible conditional branches in the ARM assembly language, including "always" (which is effectively an unconditional branch) and "never" (which is never used but exists for future possible extensions to the architecture). The complete set of branch instructions is given in the table:

Branch	Condition Test	Meaning	Uses
B	No test	Unconditional	Always take the branch
BAL	No test	Always	Always take the branch
BEQ	Z=1	Equal	Comparison equal or zero result
BNE	Z=0	Not equal	Comparison not equal or non-zero result
BCS	C=1	Carry set	Arithmetic operation gave carry out
BCC	C=1	Carry clear	Arithmetic operation did not produce a carry
BHS	C=1	Higher or same	Unsigned comparison gave higher or same result
BLO	C=0	Lower	Unsigned comparison gave lower result
BMI	N=1	Minus	Result is minus or negative
BPL	N=0	Plus	Result is positive (plus) or zero
BVS	V=1	Overflow Set	Signed integer operation: overflow occurred
BVC	V=0	Overflow Clear	Signed integer operation: no overflow occurred
BHI	((NOT C) OR Z) =0 {C set and Z clear}	Higher	Unsigned comparison gave higher
BLS	((NOT C) OR Z) =1 {C set or Z clear}	Lower or same	Unsigned comparison gave lower or same
BGE	(N EOR V) =0 {(N and V) set or (N and V) clear}	Greater or Equal	Signed integer comparison gave greater than or equal
BLT	(N EOR V) =1 {(N set and V clear) or (N clear and V set)}	Less Than	Signed integer comparison gave less than
BGT	(Z OR (N EOR V)) =0 {((N and V) set or clear) and Z clear}	Greater Than	Signed integer comparison gave greater than
BLE	(Z OR (N EOR V)) =1 {(N set and V clear) or (N clear and V set) or Z set}	Less or Equal	Signed integer comparison gave less than or equal

Special Instructions of ARM7

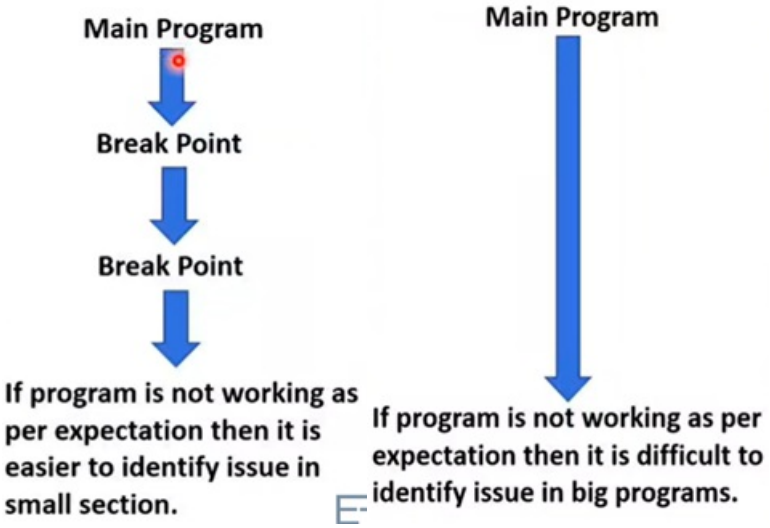
- **SWI-Software Interrupt**



  - ➢ SWI will invoke Supervisor Mode of ARM.
  - ➢ SWI can be used by programmer to invoke supervisor functions.
  - ➢ Supervisor mode is having OS functions.
  - ➢ Next to SWI instruction, we can write 24bits number, which indicates which function needs to be invoked.
  - ➢ On SWI instruction, ARM will load PC & CPSR of user mode on LR & SPSR of Supervisor Mode.
  - ➢ To come back to user mode, execute

    MOVS PC, R14_sVC
  - ➢ It restores PC & CPSR of user mode from LR & SPSR of Supervisor Mode.

- **BKPT-Break Point**
  - ➢ BKPT is used to insert debug break point in program.
  - ➢ For Simplicity, Big Programs are bisected into parts and after every part user can put break point to see the working of program.
  - ➢ So, BKPT helps user to understand program in easier way.
  - ➢ By using BKPT user can understand the working of each sections of program.



Chapter 4 “ARM7 C programming”

Main points

- Download and install Keil µVision
- Using µVision IDE
- LPC2148 (32-bit ARM7TDMI-S processor) GPIO Ports and Registers
- Introducing GPIO
- Control LED using Push Button using LPC2148
- LCD 16x2 interfacing with LPC2148 (8-bit mode)
- ADC (Analog to Digital Converter) in ARM LPC2148
- LPC2148 DAC (Digital to Analog Converter)

LPC2148 UART

Exploring ARM Development Environments and Setting Up Keil µVision

ARM processors have several development environments available, each catering to different needs. Here are a few notable ones:

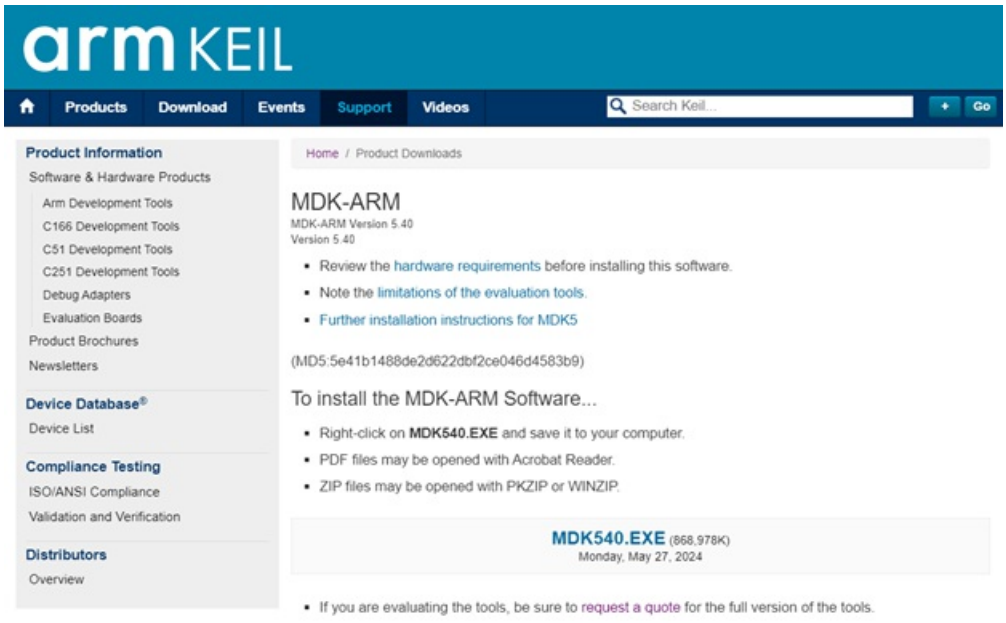
1. **CrossWorks for Arm**
2. **Keil µVision**
3. **IAR Embedded Workbench**

In this guide, we'll focus on installing and configuring Keil's µVision IDE. Follow these steps to set up the software correctly:

1. **Installation**:
    - Download and install Keil µVision from their official website.
    - Ensure you choose the appropriate version for your system (32-bit or 64-bit).
2. **Configuration**:
    - Launch µVision after installation.
    - Configure your project settings, including the target microcontroller (in our case, the LPC2148).
    - Set up the build environment, specifying memory regions, clock frequencies, and other relevant parameters.
3. **Writing a Basic LED Blinking Code**:
    - Create a new project within µVision.
    - Write a simple C program that toggles an LED connected to the LPC2148 GPIO pins.
    - Compile the code to generate the binary file.
4. **Flashing the Microcontroller**:
    - Use a suitable programmer (such as JTAG or SWD) to flash the compiled binary onto the LPC2148.
    - Verify that the LED blinks according to your program.

Download and install Keil µVision

1. **Get the MDK-Lite (Microcontroller Development Kit) from Keil's website**:
    a. You can download the MDK-Lite from Keil's official website. Visit this link: Keil MDK-Lite Download

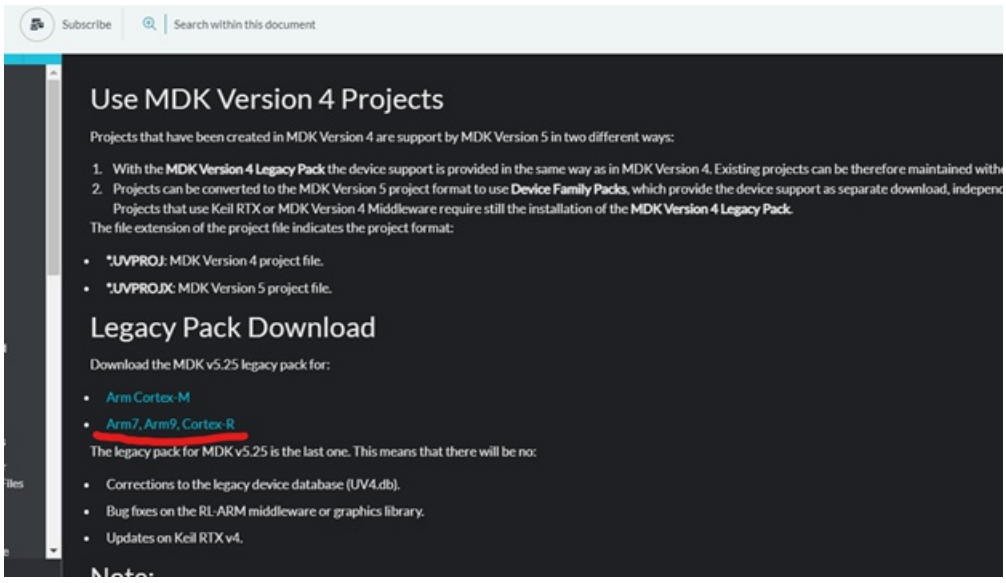


2. The latest version of µVision5 does not currently support several devices that were compatible with older versions. Unfortunately, the LPC2148 is one such unsupported device. However, we can address this by adding support manually after installing µVision5.

Follow these steps:

a) Visit the following link: Legacy Support for ARM7, ARM9, and Cortex-R.

b) Download the executable file specifically designed for legacy support corresponding to the version of MDK (µVision) you have installed.



Install the Downloaded Executable:

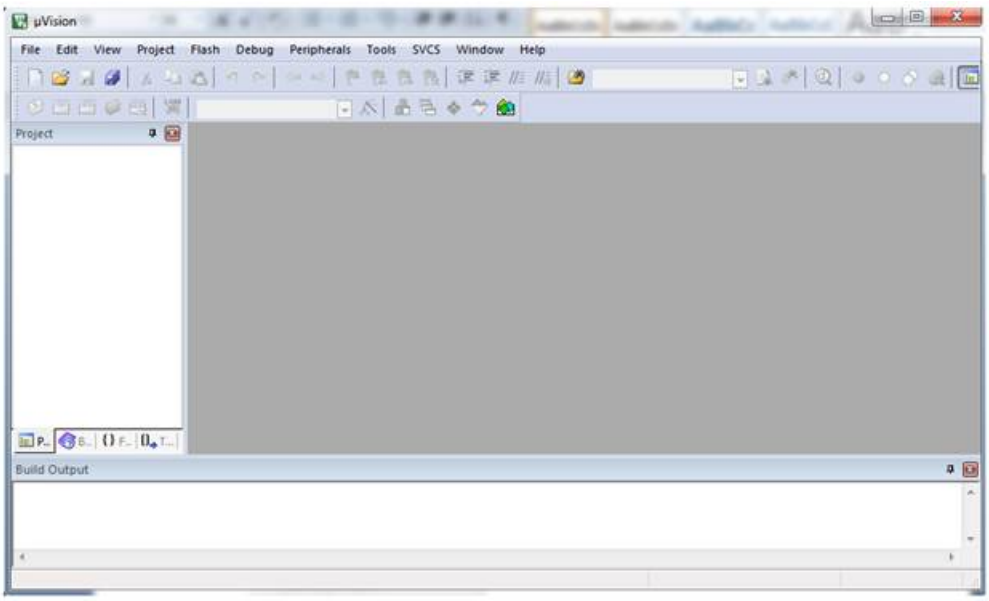
- After downloading the executable file, proceed with the installation.
- Follow the straightforward instructions provided during the installation process.

Once you've completed these steps, the IDE will be installed and ready for use, including support for the specific device (LPC2148) you intend to work with.

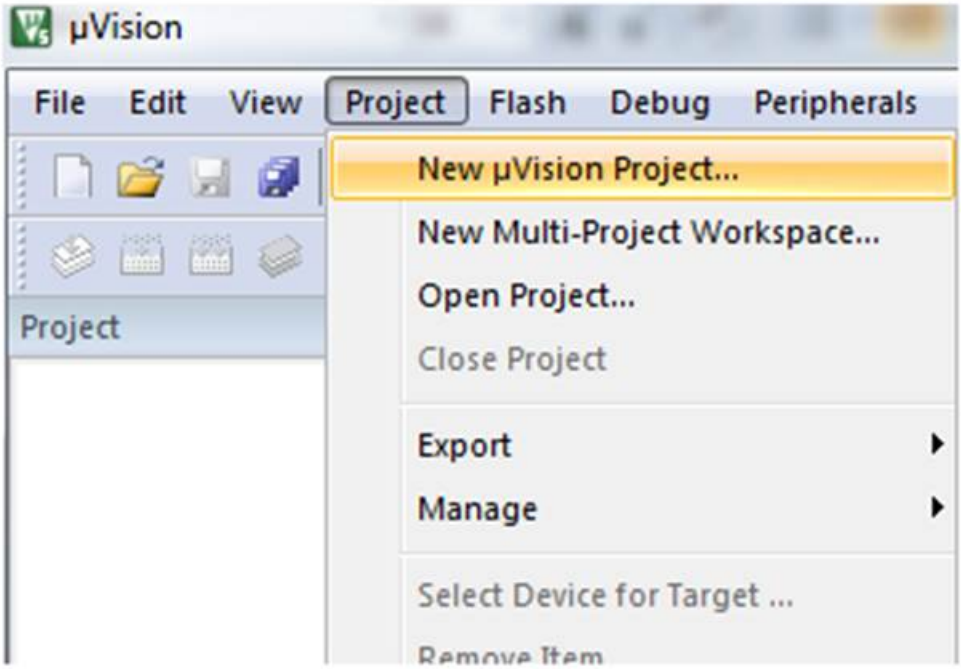
Using µVision IDE

We're about to create a straightforward LED blinking project. Here are the steps for setting up and building the project using the Keil µVision IDE:

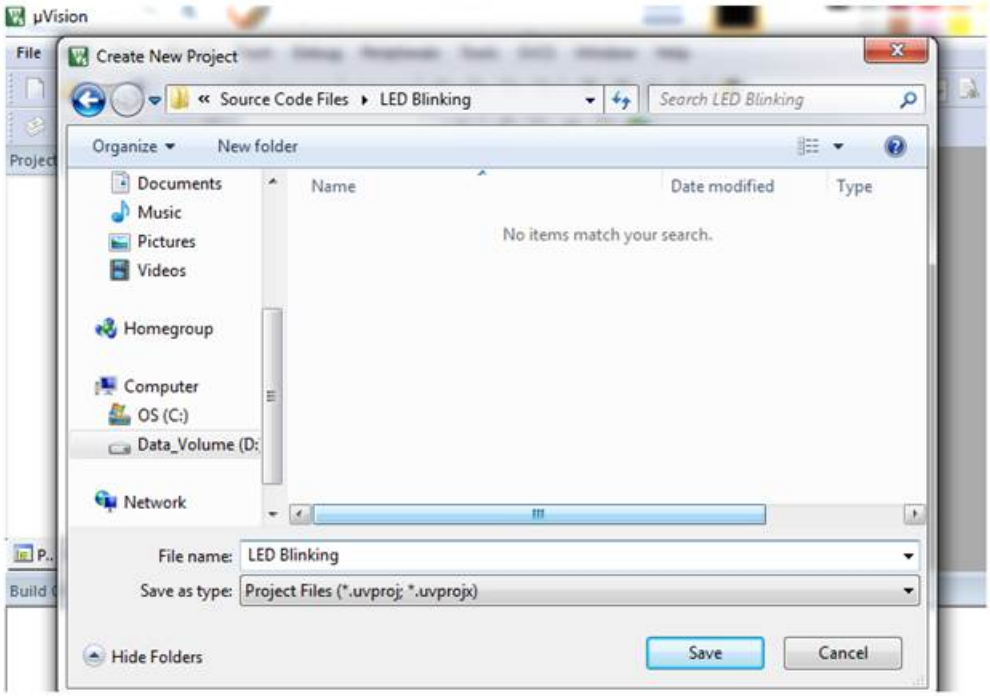
1. Open Keil µVision by clicking on the desktop icon."



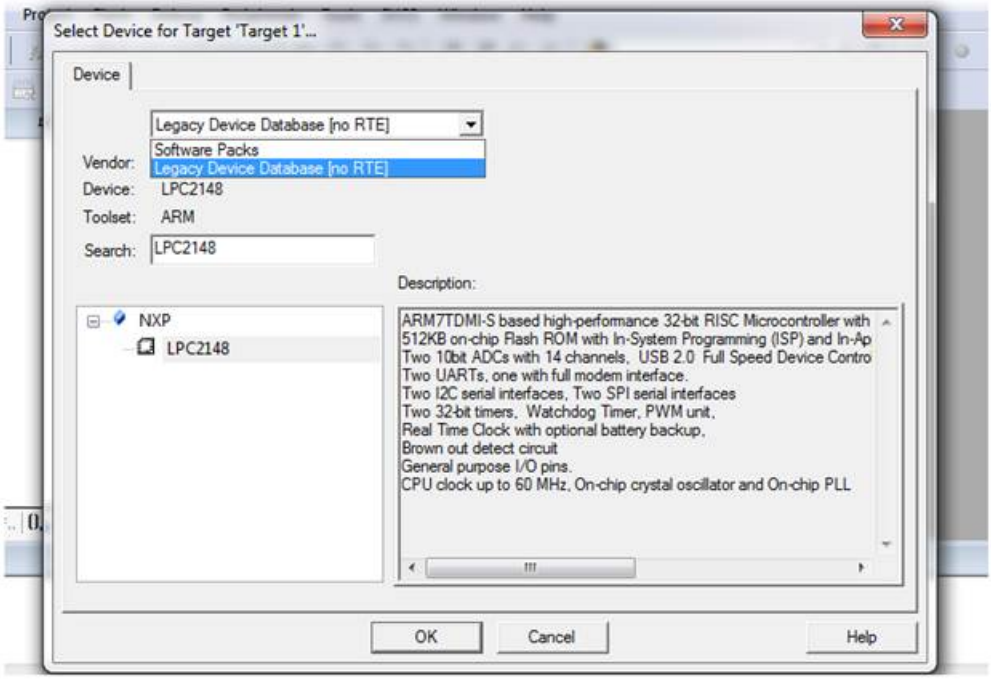
2. Go to the **Project** tab. Select New **µVision Project ...**from that menu.



3. **Create New Project** window will pop up. Select the folder where you want to create project and give a suitable name to the project. Then click on **Save**.



4. When you encounter the 'Select Device for Target: Target1' window, you'll have the option to choose between Software Packs or the Legacy Device Database. Since the LPC2148 is part of the Legacy Device Database, go ahead and select that option.

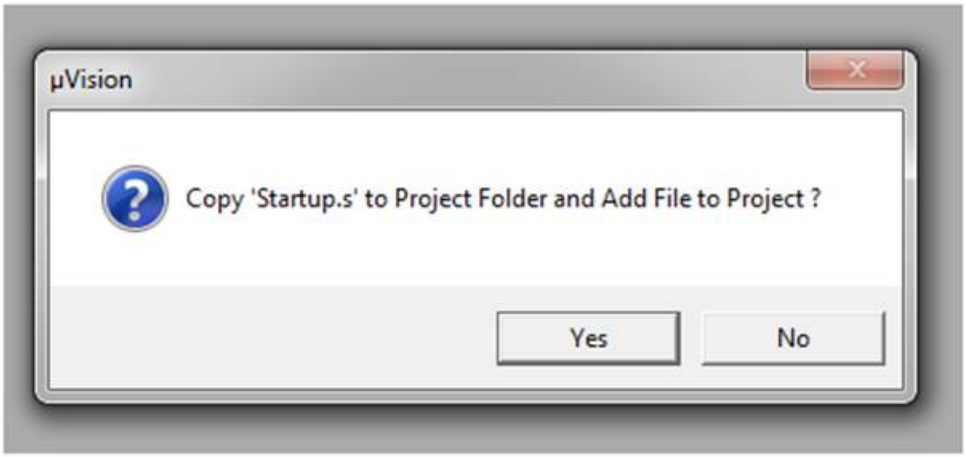


Search for LPC2148:

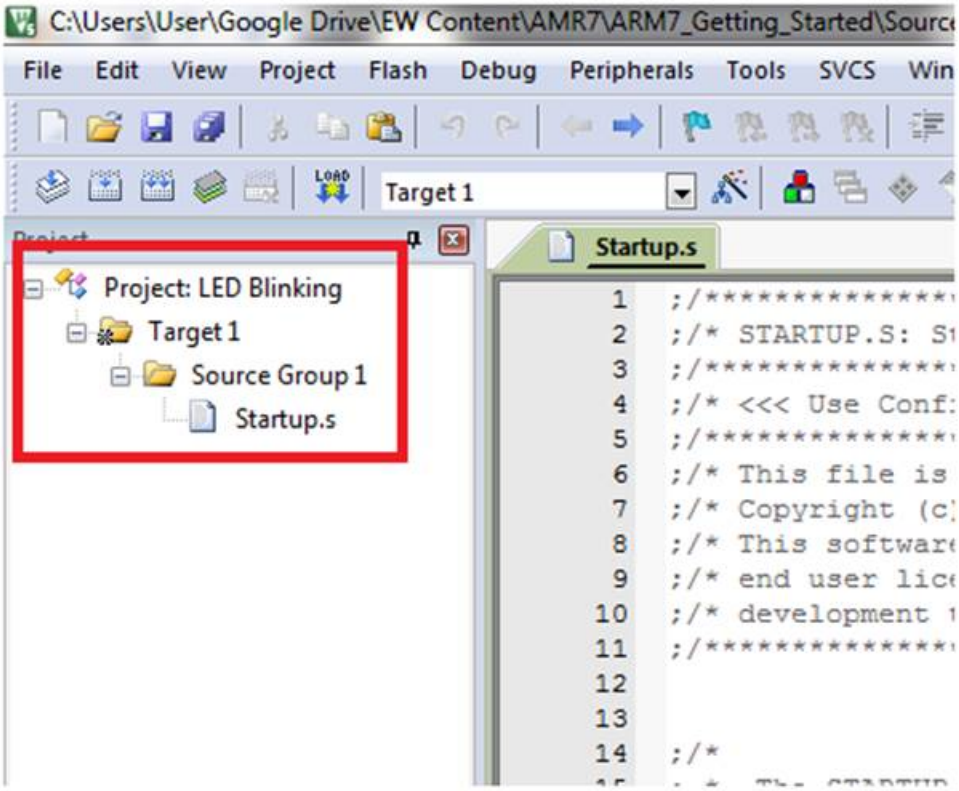
- In the search bar, type "LPC2148."
- Look for the device under NXP with the name "LPC2148" and select it.
- Click "OK."

2. **Startup.s File Prompt**:
   - A window will appear, asking whether to copy the "Startup.s" file to the project folder and add it to the project.

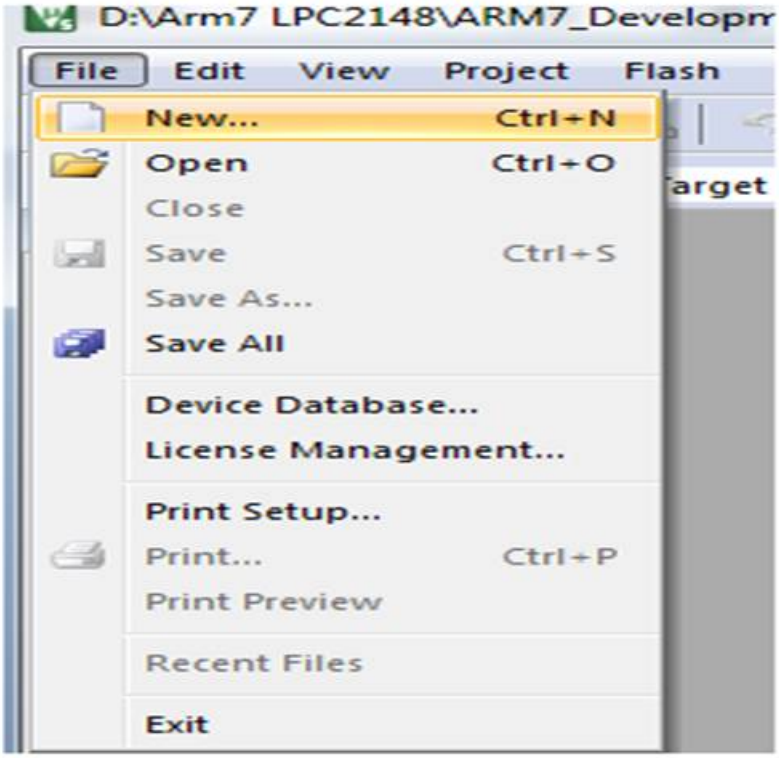
- Click “Yes.”



6. After completing the previous step, you’ll find the project name and its associated folders displayed on the left side of the project window, as illustrated below.

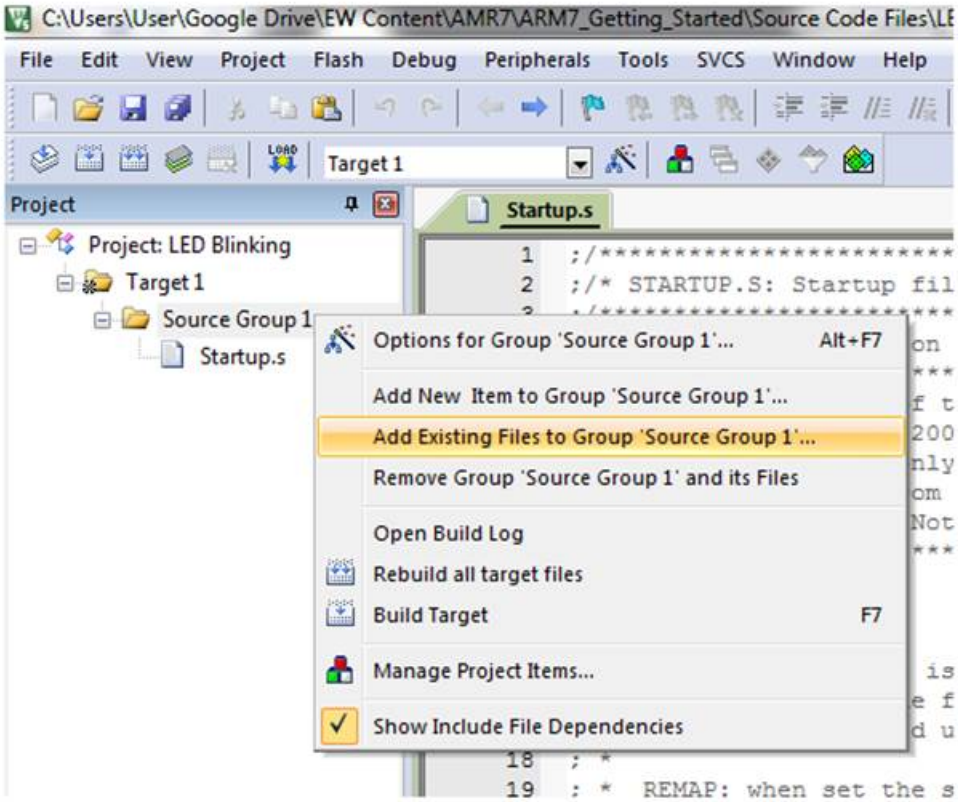


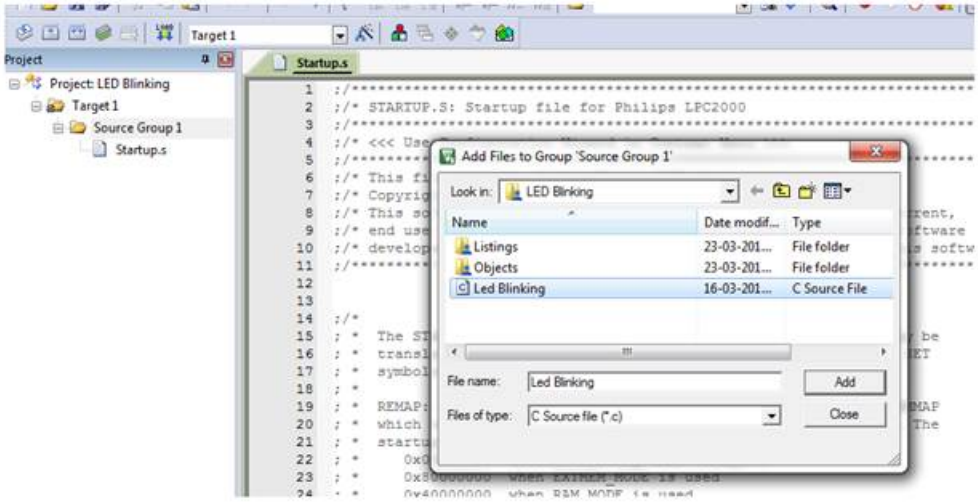
7. Now go to File tab and add **New** file from the menu.



8. After completing the previous step, save the file with a specific name and append the .c extension to the file name

9. Add this file to Source Group folder in the project window by right clicking on Source Group1 folder and selecting **Add Existing Files to Group 'Source Group1'...**



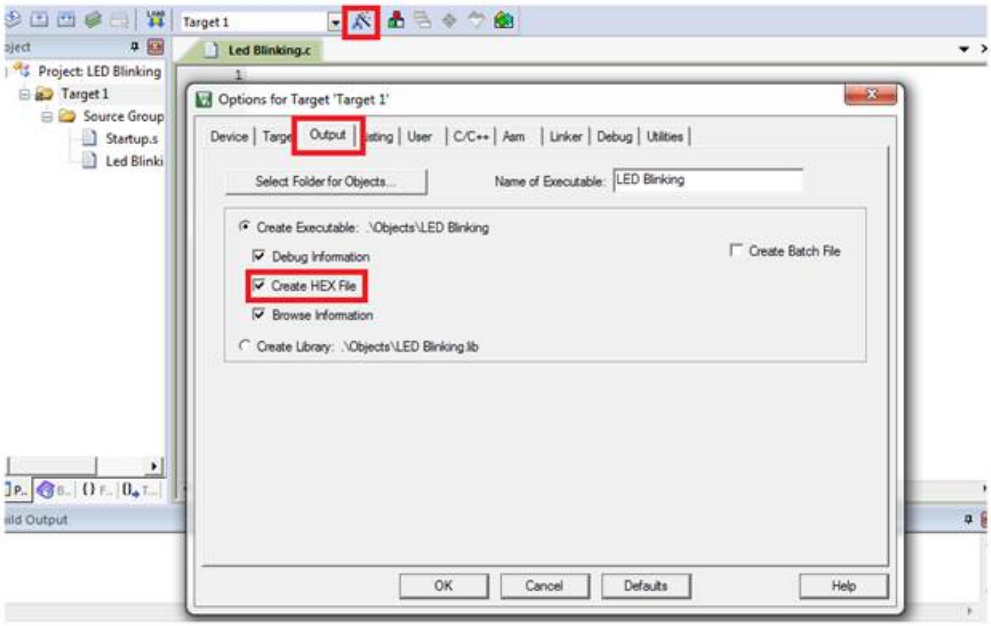


10. Select the Previously Saved File:

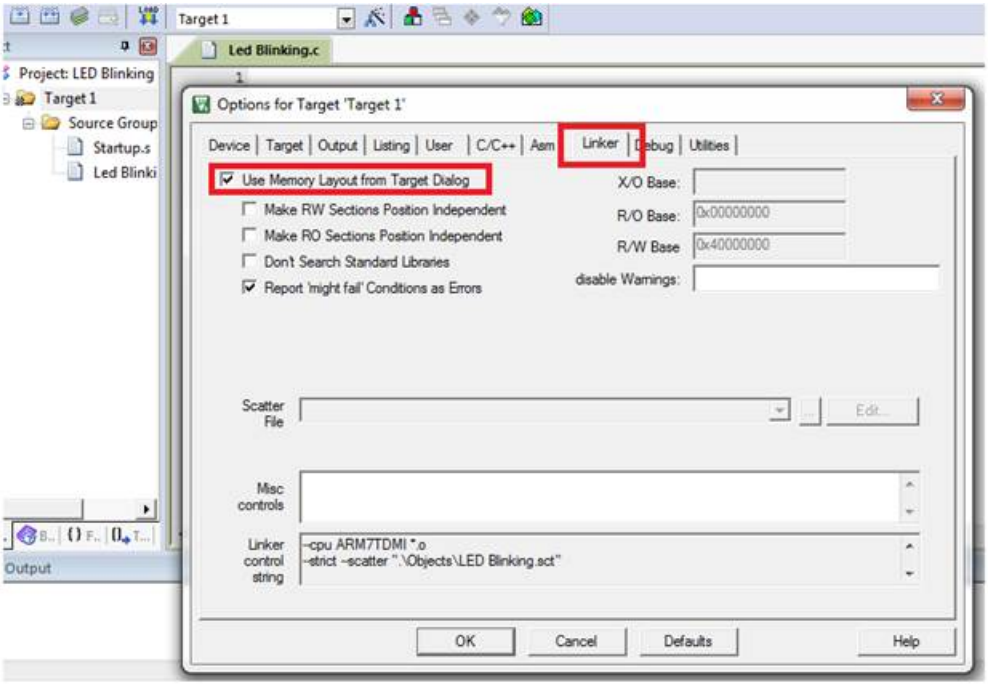
- Open the window that pops up
- Locate the file you saved earlier.
- Add this file to “Source Group1.”
- In our specific case, the file is named “LED Blinking.c.”

1. **Configure Target Options:**
   - Look for the symbol (usually an icon) representing “Options for Target ‘Target1’.” It’s highlighted in a red box in the image below.
   - Alternatively, you can press Alt+F7 or right-click on “Target1” and choose “Options for Target ‘Target1’.”
   - The “Options for target” window will appear.

- Navigate to the “Output” tab within that window.
- Check the box next to “Create HEX File.” This step is crucial because we need the HEX file to burn it into the microcontroller.



2. Open the “Options for Target” window. Navigate to the “Linker” tab. Look for the “Use Memory Layout from Target Dialogue” option and select it.



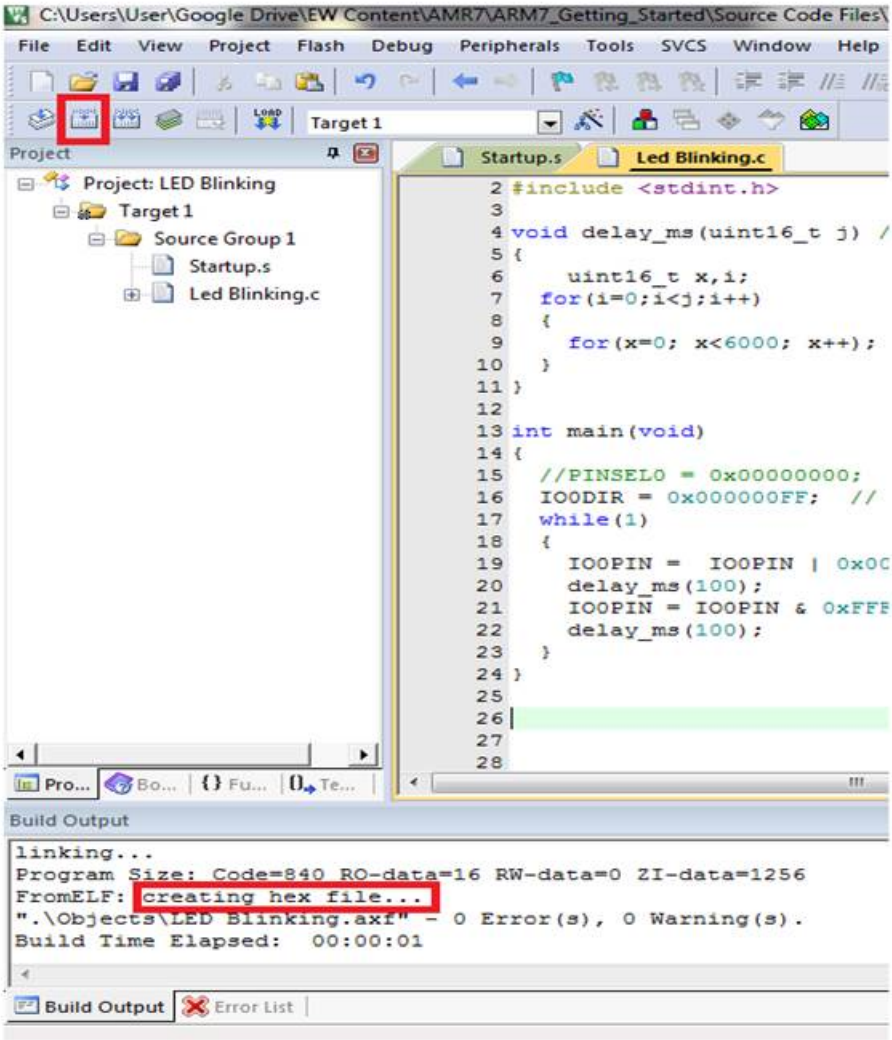
Then click on OK.

13. Now write the code for LED Blinking.

```
/*
  LED Blinking on LPC2148(ARM7)
```

```
   http://www.electronicwings.com/arm7/getting-started-with-arm-lpc2148-using-keil-uvision-ide
*/

#include <lpc214x.h>
#include <stdint.h>

void delay_ms(uint16_t j) /* Function for delay in milliseconds */
{
   uint16_t x,i;
            for(i=0;i<j;i++)
            {
   for(x=0; x<6000; x++);    /* loop to generate 1 millisecond delay with 12MHz Fosc. */
            }
}

int main(void)
{
            IO0DIR = 0x000000FF;       /* Set P0.0 to P0.7 bits as output bits by writing 1 in IO0DIR register corresponding to those bits.
*/
            while(1)
            {
                        IO0PIN =  IO0PIN | 0x000000FF;
                        /* Make P0.0 to P0.7 HIGH while keeping other bits unchanged. */
                        delay_ms(300);
                        IO0PIN = IO0PIN & 0xFFFFFF00;
                         /* Make P0.0 to P0.7 LOW while keeoing other bits unchanged. */
                        delay_ms(300);
            }
}
```

14. **Build Your Code:**
    - After writing your code, click the button highlighted in red in the image below.
    - Alternatively, you can build the project by choosing “Build Target” from the “Project” tab or simply pressing F7 on your keyboard.



You can see **creating hex file ...** in the Build Output window as shown in the image.

LPC2148 (32-bit ARM7TDMI-S processor) GPIO Ports and Registers

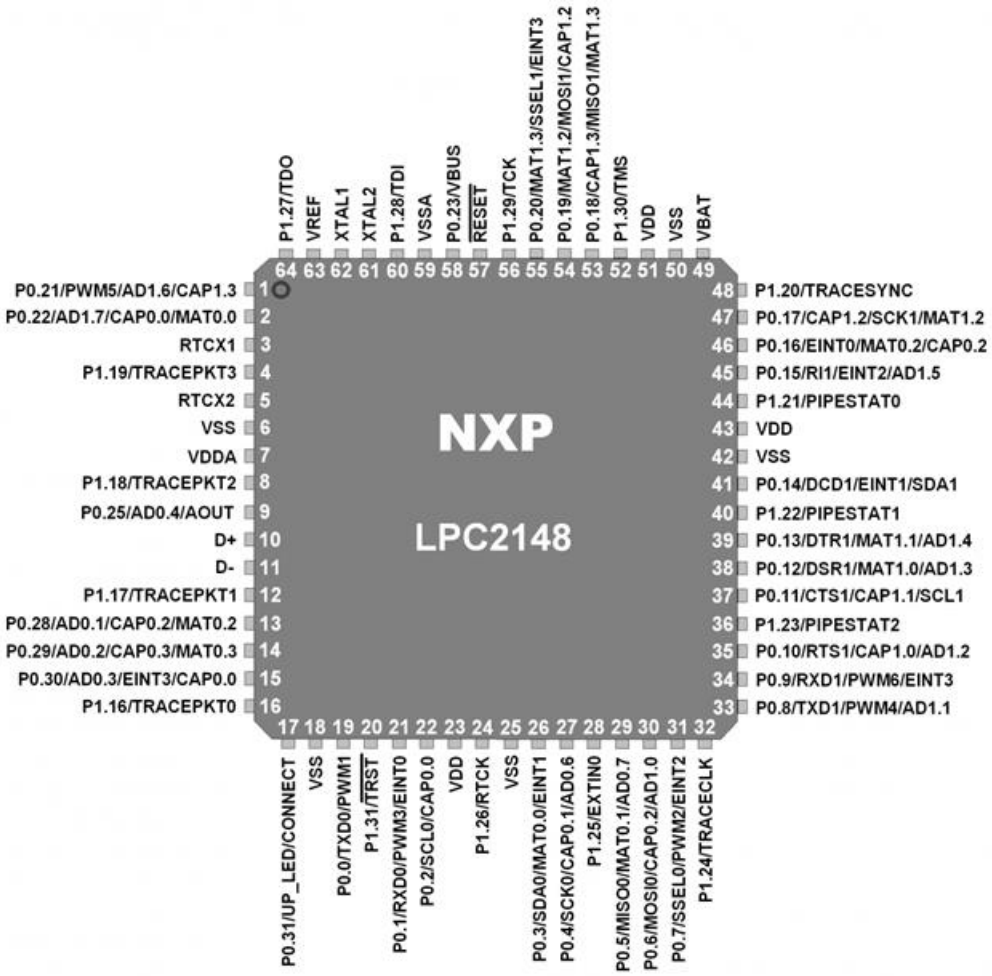
- Introducing GPIO
- Pin Function Select Registers
- Fast and Slow GPIO Registers
- Control LED using Push Button using LPC2148

Introducing GPIO

Let's break down the information about general-purpose input/output (GPIO) and the LPC2148 microcontroller:

1. **GPIO Basics:**
    - GPIO stands for "General-Purpose Input/Output."
    - It refers to pins on an Integrated Circuit (IC) that can be configured either as input pins or output pins.
    - The behavior of these pins can be controlled dynamically during runtime.
    - A group of these pins is collectively called a "port."
2. **LPC2148 and Its GPIO Ports:**
    - The LPC2148 microcontroller features two 32-bit General Purpose I/O ports: PORT0 and PORT1.
    - Let's dive into each of these ports:
        - **PORT0:**
            - PORT0 is a 32-bit port.
            - Out of these 32 pins, 28 pins can be configured as either general-purpose input or output.
            - However, one specific pin (P0.31) can only be configured as a general-purpose output.
            - Interestingly, three pins (P0.24, P0.26, and P0.27) are reserved and not available for use. These pins are not even mentioned in the pin diagram.
        - **PORT1:**
            - PORT1 is also a 32-bit port.
            - However, only 16 pins within this port (P1.16 – P1.31) are available for use as general-purpose input or output.

ARM LPC2148 Pin Diagram



1. **Alternate Functions for GPIO Pins:**
   - Almost every pin in PORT0 and PORT1 has additional alternate functions beyond simple input/output.
   - For instance, consider P0.0:
     - It can serve as the TXD (transmit data) pin for UART0.
     - Alternatively, it can function as a PWM1 (pulse-width modulation) pin.
   - The specific functionality of each pin is determined by the Pin Function Select Registers.
2. **External Resistors for PORT0:**
   - Here's an important note: PORT0 pins lack built-in pull-up or pull-down resistors.
   - When using GPIOs on PORT0, especially in certain cases, you'll need to connect external pull-up or pull-down resistors.
   - These external resistors help ensure stable behavior and prevent floating states.

Pin Function Select Registers

Pin Function Select Registers are 32-bit registers. These registers are used to select or configure specific pin functionality.

There are 3 Pin Function Select Registers in LPC2148:

1. **PINSEL0 : -** PINSEL0 is used to configure PORT0 pins P0.0 to P0.15.
2. **PINSEL1 : -** PINSEL1 is used to configure PORT0 pins P0.16 to P0.31.
3. **PINSEL2 : -** PINSEL2 is used to configure PORT1 pins P1.16 to P1.31.

Let's see GPIO registers that control the GPIO operations.

Fast and Slow GPIO Registers

1. **Fast (Enhanced) GPIO Registers:**
    - There are five of these registers.
    - They provide advanced features for controlling GPIO pins.
    - Think of them as the speed demons—optimized for modern functionality.
2. **Slow (Legacy) GPIO Registers:**
    - We have four of these registers.
    - They serve as a bridge to the past, maintaining compatibility with earlier family devices.
    - Imagine them as the wise elders—still relevant, even in the fast-paced digital world.

Slow GPIO Registers

There are 4 Slow GPIO registers :

1. **IOxPIN (GPIO Port Pin value register):** This is a 32-bit wide register. this 32-bit register:
    - It serves as a communication channel for reading from or writing to the Port (either PORT0 or PORT1).
    - When writing to this register, caution is essential. We need to be precise about which pin we're targeting.
    - To ensure accuracy, masking techniques are employed.

Examples:

a) Writing 1 to P0.4 using IO0PIN

```
IO0PIN = IO0PIN | (1<<4)
```

b) Writing 0 to P0.4 using IO0PIN

```
IO0PIN = IO0PIN & (~(1<<4) )
```

c) Writing F to P0.7-P0.4

```
IO0PIN = IO0PIN | (0x000000F0)
```

2. **IOxSET (GPIO Port Output Set register) :** This is a 32-bit wide register. This register is used to make pins of Port (PORT0/PORT1) HIGH. Writing one to specific bit makes that pin HIGH. Writing zero has no effect.

3. **IOxDIR (GPIO Port Direction control register) :** This is a 32-bit wide register.
    - This register allows individual control over the direction of each port pin (whether it's an input or an output).

- When you set a bit to '1' in this register, it configures the corresponding pin as an output.
- Conversely, setting a bit to '0' designates the corresponding pin as an input.

4. **IOxCLR (GPIO Port Output Clear register) :** This is a 32-bit wide register. This register is used to make pins of Port LOW. Writing one to specific bit makes that pin LOW. Writing zeroes has no effect.

Examples:

a) Configure pin P0.0 to P0.3 as input pins and P0.4 to P0.7 as output pins.

```
IO0DIR = 0x000000F0;
```

b) Configure pin P0.4 as an output. Then set that pin HIGH.

```
IO0DIR = 0x00000010; OR IO0DIR = (1<<4);
IO0SET = (1<<4);
```

c) Configure pin P0.4 as an output. Then set that pin LOW.

```
IO0DIR = 0x00000010; OR IO0DIR = (1<<4);
IO0CLR = (1<<4);
```

Fast GPIO Registers

There are 5 fast GPIO registers :

1. **FIOxDIR (Fast GPIO Port Direction Control Register):**
   - This 32-bit wide register allows individual control over the direction of each port pin.
   - Setting a bit to '1' configures the corresponding pin as an output.
   - Conversely, setting a bit to '0' designates the corresponding pin as an input.
2. **FIOxMASK (Fast Mask Register for Port):**
   - Also a 32-bit wide register, FIOxMASK controls the effect of other fast registers (such as FIOxPIN, FIOxSET, and FIOxCLR) on port pins.
   - When you set a bit to '0' in FIOxMASK, it enables access to the fast registers for the corresponding pin. In other words, you can read from or write to that pin in fast mode using the fast registers.
   - Setting a bit to '1' in FIOxMASK ensures that the corresponding pin remains unaffected by the fast registers.
3. **FIOxPIN (Fast Port Pin Value Register using FIOMASK):**
   - This 32-bit wide register is used to read from or write to the value on port pins.
   - However, access to this register is only allowed for those port pins that have been enabled using zeroes in FIOxMASK.

4. **FIOxSET (Fast Port Output Set Register using FIOMASK):**
    - Another 32-bit wide register, FIOxSET is used to set pins of the port to a HIGH state.
    - Writing a '1' to a specific bit makes that pin HIGH.
    - Reading this register returns the current contents of the port output register.
    - Note that only bits enabled by zeroes in FIOMASK can be altered.
5. **FIOxCLR (Fast Port Output Clear Register using FIOMASK):**
    - Like FIOxSET, this 32-bit wide register is used to clear pins of the port, making them LOW.
    - Writing a '1' to a specific bit makes that pin LOW.
    - Only bits enabled by zeroes in FIOMASK can be altered.

Aside from the 32-bit long and word only accessible registers mentioned above, every fast GPIO port can also be controlled via several byte and half-word accessible registers. Refer chapter 8 (Page 83) on GPIO in the datasheet of LPC2148 provided in the attachments section for their use.

Examples:

a) Configure pin P0.0 to P0.3 as input pins and P0.4 to P0.7 as output pins.

```
FIO0DIR = 0x000000F0;
```

b) Configure pin P0.4 as an output. Then set that pin HIGH.

```
FIO0DIR = 0x00000010; OR FIO0DIR = (1<<4);
FIO0SET = (1<<4);
```

c) Configure pin P0.4 as an output. Then set that pin LOW.

```
FIO0DIR = 0x00000010; OR FIO0DIR = (1<<4);
FIO0CLR = (1<<4);
```

Why this is called Fast GPIO :

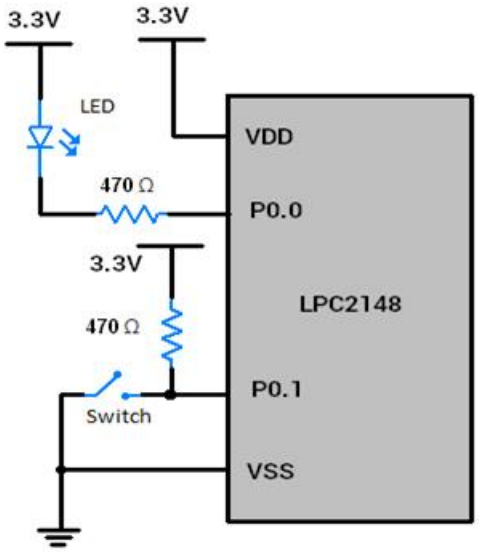
Fast GPIO registers are relocated to the ARM local bus. This makes software access to GPIO pins 3.5 times faster than access through Slow GPIO registers. This effect will not always be visible when a program is written in c code. It may be more evident in an assembly code than in a c code.

Control LED using Push Button using LPC2148

Let’s create a straightforward program to control an LED based on the status of a pin. Here’s the plan:

1. LED and Pin Setup:
    - We have an LED connected to pin P0.0.
    - Additionally, there’s a switch connected to pin P0.1. This switch will allow us to change the pin’s status.
2. Program Logic:
    - If the switch is pressed (or closed), we’ll turn the LED ON.
    - If the switch is not pressed (open), we’ll turn the LED OFF.

Interfacing Diagram



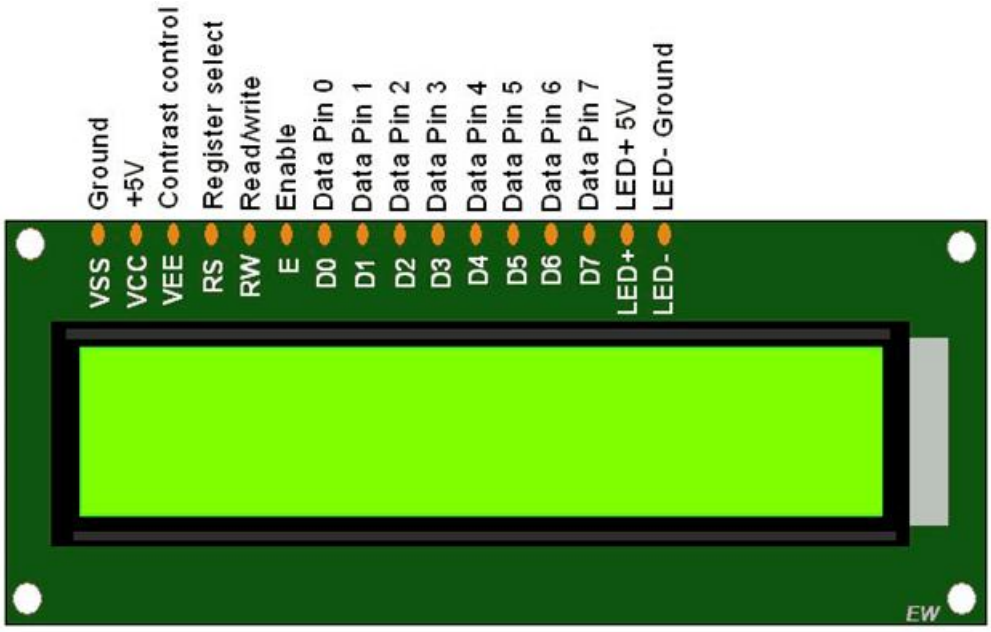
Program for Control LED using Push Button

```
/*
  Input-Output Pin Programming to Drive LED through switch
  http://www.electronicwings.com/arm7/lpc2148-32-bit-arm7tdmi-s-processor-gpio-ports-and-registers
*/

#include <lpc214x.h>
#include <stdint.h>

int main(void)
{
        //PINSEL0 = 0x00000000;             /* Configuring P0.0 to P0.15 as GPIO */
        /* No need for this as PINSEL0 reset value is 0x00000000 */
```

```
        IO0DIR = 0x00000001;
        /* Make P0.0 bit as output bit, P0.1 bit as an input pin  */
        while(1)
        {
                if ( IO0PIN & (1<<1) )     /* If switch is open, pin is HIGH */
                {
                        IO0CLR = 0x00000001;   /* Turn on LED */
                }
                else  /* If switch is closed, pin is LOW */
                {
                        IO0SET = 0x00000001;     /* Turn off LED */
                }
        }
}
```

LCD 16x2 interfacing with LPC2148 (8-bit mode)

Introduction



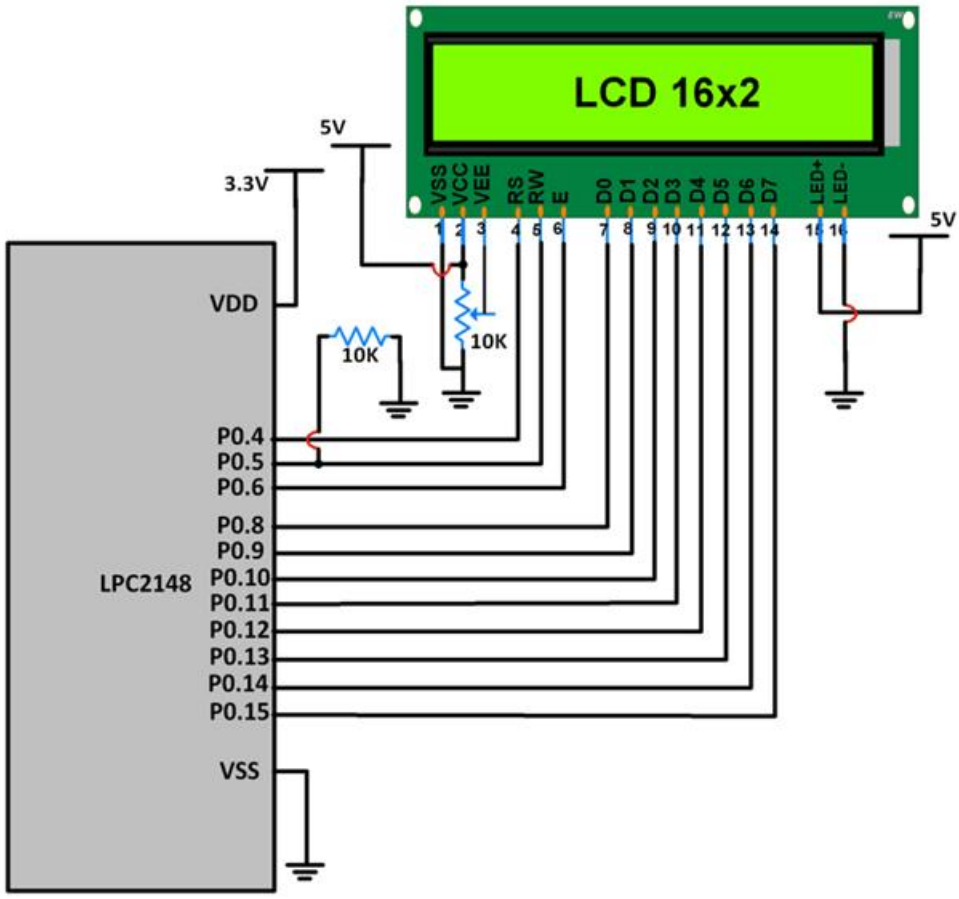
LCD 16x2 Display

the essentials about the 16x2 LCD (Liquid Crystal Display):

1. **What's an LCD 16x2?**
   - The 16x2 LCD is a common display module used in embedded systems.
   - It has 16 pins in total.
   - Among these pins:
     - **Data Pins (D0-D7):** These eight pins handle data communication.
     - **Control Pins:**
       - **RS (Register Select):** Determines whether we're sending a command or data to the LCD.
       - **RW (Read/Write):** Configures read or write mode.
       - **EN (Enable):** Signals when data is ready to be read or written.
     - The remaining five pins handle power supply and backlighting.
2. **Modes and Configuration:**
   - The control pins allow us to configure the LCD:
     - **Command Mode:** Used for sending configuration commands to the LCD (e.g., setting display properties).
     - **Data Mode:** Used for sending actual data to display (e.g., characters or custom symbols).
     - We can choose between 4-bit mode or 8-bit mode based on our application's needs.
3. **How It Works:**

- First, we send specific commands to configure the LCD (command mode).
- Once set up, we switch to data mode and send the desired data (text, numbers, etc.) to display on the screen.

Interfacing Diagram



Interfacing 16x2 LCD With LPC2148

LCD Data pins D0-D7 are connected to pins P0.8-P0.15 of Port 0.

LCD control pins RS, RW and EN are connected to P0.4, P0.5 P0.6 respectively.

LCD programming

1. Initializing the LCD

```
void LCD_INIT(void)
{
        IO0DIR = 0x0000FFF0; /* P0.8 to P0.15 LCD Data. P0.4,5,6 as RS RW and EN */
        delay_ms(20);
        LCD_CMD(0x38);  /* Initialize LCDcommand */
        LCD_CMD(0x0C);   /* Display on cursor off */
        LCD_CMD(0x06);  /* Auto increment cursor */
        LCD_CMD(0x01);   /* Display clear */
        LCD_CMD(0x80);  /* First line first position */
```

```
}
```

2. Command Write Function

- For command mode, RS = 0.
- RW = 0 for write.
- High to low EN pulse of minimum 450ns high pulse width.

```
void LCD_CMD(char command)
{
        IO0PIN = ( (IO0PIN & 0xFFFF00FF) | (command<<8) );
        IO0SET = 0x00000040; /* EN = 1 */
        IO0CLR = 0x00000030; /* RS = 0, RW = 0 */
        delay_ms(2);
        IO0CLR = 0x00000040; /* EN = 0, RS and RW unchanged(i.e. RS = RW = 0) */
        delay_ms(5);
}
```

3. **Data Write Function (Single Character)**

- For data mode, RS = 1.
- RW = 0 for write.
- High to low EN pulse of minimum 450ns high pulse width.

```
void LCD_CHAR (char msg)
{
                IO0PIN = ( (IO0PIN & 0xFFFF00FF) | (msg<<8) );
                IO0SET = 0x00000050; /* RS = 1, , EN = 1 */
                IO0CLR = 0x00000020; /* RW = 0 */
                delay_ms(2);
                IO0CLR = 0x00000040; /* EN = 0, RS and RW unchanged(i.e. RS = 1, RW = 0) */
                delay_ms(5);
}
```

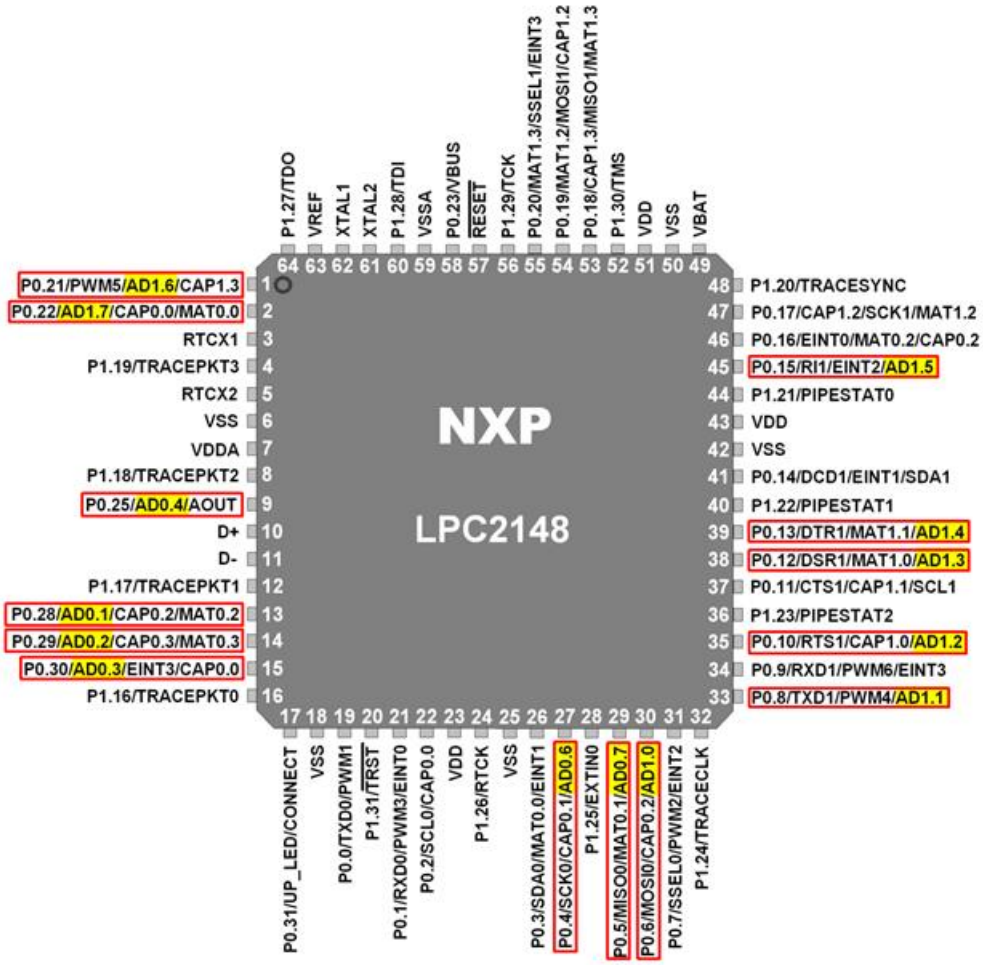
ADC (Analog to Digital Converter) in ARM LPC2148

Introduction to ADC

Let's dive into the world of analog-to-digital conversion (ADC) and explore LPC2148's built-in ADC capabilities:

1. **ADC Basics:**
   - An Analog-to-Digital Converter (ADC) is like a bilingual translator—it converts analog signals (continuous voltage levels) into their digital counterparts (discrete values).
   - In our case, the LPC2148 microcontroller boasts not one but **two** 10-bit ADCs: ADC0 and ADC1.
2. **Channel Chatter:**
   - ADC0 has **6 channels**, while ADC1 flaunts **8 channels**.
   - What's a channel? Think of it as a dedicated input path for a specific analog signal. So, we can connect different types of sensors or signals to these channels.
3. **Successive Approximation Magic:**
   - The LPC2148 ADCs use the **Successive Approximation** technique.
   - It's like playing "20 Questions" with your microcontroller: it keeps narrowing down the possible values until it zeroes in on the right one.
4. **Clocking In:**
   - To perform this magic, the ADC needs a clock. But not just any clock—it should be **4.5 MHz or less**.
   - We can adjust this clock using clock divider settings. Imagine it as tuning the ADC's heartbeat.
5. **Voltage Range Dance:**
   - When converting analog signals, both ADCs dance within the voltage range of **0V to VREF**.
   - Typically, VREF is around **3V**, but we mustn't exceed the microcontroller's supply voltage (VDDA).

ARM LPC4128 ADC Pins



AD0.1:4, AD0.6:7 & AD1.7:0 (Analog Inputs)

These are Analog input pins of ADC. If ADC is used, signal level on analog pins must not be above the level of VDDA; otherwise, ADC readings will be invalid. If ADC is not used, then the pins can be used as 5V tolerant digital I/O pins.

VREF (Voltage Reference)

Provide Voltage Reference for ADC.

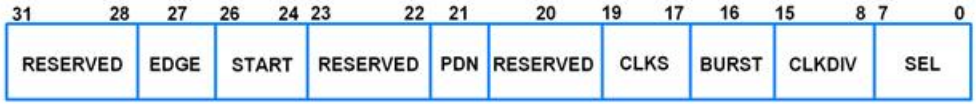
VDDA& VSSA (Analog Power and Ground)

These are the power and ground pins for ADC. These should be same as VDD & VSS.

Let's explore the ADC0 registers and their configurations. Remember, ADC1 follows a similar pattern. Here's the rundown:

1. **AD0CR (ADC0 Control Register):**
   - AD0CR is a 32-bit register—the control center for ADC0.
   - Before any A/D conversion magic happens, we must write to this register to set the operating mode.
   - What can we configure with AD0CR?
     - **Channel Selection:** Choose which ADC channel to read from.
     - **Clock Frequency:** Set the clock speed for ADC operations.
     - **Result Precision:** Specify the number of bits in the conversion result.

- **Start of Conversion:** Trigger the A/D conversion process.
- And a few other handy parameters.



Let's dive into the details of the ADC0 Control Register (AD0CR) for LPC2148. Buckle up—we're decoding bits and bytes!

a) Channel Selection (Bits 7:0 - SEL):
   - These bits allow us to choose the ADC0 channel for analog input.
   - In software-controlled mode, only one of these bits should be set to 1.
   - For example, setting bit 7 (10000000) selects AD0.7 channel as the analog input.

b) Clock Division (Bits 15:8 - CLKDIV):
   - The APB (ARM Peripheral Bus) clock is divided by this value plus one to produce the ADC clock.
   - This ADC clock should be less than or equal to 4.5 MHz.

c) Burst Mode (Bit 16 - BURST):
   - 0 = Conversions are software-controlled and require 11 clocks.
   - 1 = In Burst mode, ADC performs repeated conversions at the rate selected by the CLKS field.
   - Burst mode can be terminated by clearing this bit, but ongoing conversions will complete.
   - Note: When Burst = 1, the START bits must be 000; otherwise, conversions won't start.

d) Clocks and Result Accuracy (Bits 19:17 - CLKS):
   - CLKS selects the number of clocks used for each conversion in Burst mode.
   - It also determines the number of bits of accuracy in the result (stored in AD0DR).
   - Examples:
     - 000 = 11 clocks / 10 bits
     - 001 = 10 clocks / 9 bits
     - **010** = 9 clocks / 8 bits
     - **011** = 8 clocks / 7 bits
     - **100** = 7 clocks / 6 bits
     - **101** = 6 clocks / 5 bits
     - **110** = 5 clocks / 4 bits
     - **111** = 4 clocks / 3 bits

e) **Bit 20 –** RESERVED

f) Power Down (Bit 21 - PDN):
   - 0 = ADC is in Power Down mode.
   - 1 = ADC is operational.

g) Start Conversion (Bits 26:24 - START):
   - When BURST bit is 0, these bits control when A/D conversion starts:
     - 000 = No start (use when clearing PDN to 0)
     - 001 = Start conversion now

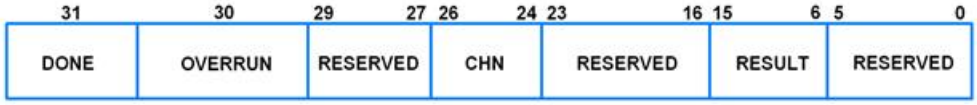
- **010** = Start conversion when edge selected by bit 27 of this register occurs on CAP0.2/MAT0.2 pin
- **011**= Start conversion when edge selected by bit 27 of this register occurs on CAP0.0/MAT0.0 pin
- **100** = Start conversion when edge selected by bit 27 of this register occurs on MAT0.1 pin
- **101** = Start conversion when edge selected by bit 27 of this register occurs on MAT0.3 pin
- **110** = Start conversion when edge selected by bit 27 of this register occurs on MAT1.0 pin
- **111** = Start conversion when edge selected by bit 27 of this register occurs on MAT1.1 pin

h) Edge Selection (Bit 27 - EDGE):
- Significant only when START field contains 010-111.
- 0 = Start conversion on a rising edge of the selected CAP/MAT signal.
- 1 = Start conversion on a falling edge of the selected CAP/MAT signal.

i) Reserved Bits (31:28)

2. **AD0GDR (ADC0 Global Data Register)**

- AD0GDR is a 32-bit register.
- This register contains the ADC's DONE bit and the result of the most recent A/D conversion.



AD0GDR (ADC0 Global Data Register)

- **Bit 5:0 – RESERVED**
- **Bits 15:6 – RESULT**
  When **DONE** bit is set to 1, this field contains 10-bit ADC result that has a value in the range of 0 (less than or equal to VSSA) to 1023 (greater than or equal to VREF).
- **Bit 23:16 – RESERVED**
- **Bits 26:24 – CHN**
  These bits contain the channel from which ADC value is read.
  e.g. 000 identifies that the **RESULT** field contains ADC value of channel 0.
- **Bit 29:27 – RESERVED**
- **Bit 30 – Overrun**
  This bit is set to 1 in burst mode if the result of one or more conversions is lost and overwritten before the conversion that produced the result in the **RESULT** bits.
  This bit is cleared by reading this register.
- **Bit 31 – DONE**
  This bit is set to 1 when an A/D conversion completes. It is cleared when this register is read and

when the AD0CR is written.
If AD0CR is written while a conversion is still in progress, this bit is set and new conversion is started.

3. ADGSR (A/D Global Start Register)

- ADGSR is a 32-bit register.
- Software can write to this register to simultaneously start conversions on both ADC.

ADGSR (A/D Global Start Register)

- **BURST (Bit 16), START (Bit <26:24>) & EDGE (Bit 27)**
  These bits have same function as in the individual ADC control registers i.e. AD0CR & AD1CR. Only difference is that we can use these function for both ADC commonly from this register.

4. AD0STAT (ADC0 Status Register)

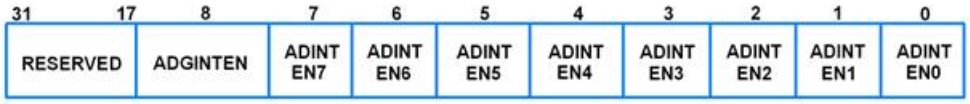
- AD0STAT is a 32-bit register.
- It allows checking of status of all the A/D channels simultaneously.

AD0STAT (ADC0 Status Register)

- **Bit 7:0 – DONE7:DONE0**
  These bits reflect the **DONE** status flag from the result registers for A/D channel 7 - channel 0.
- **Bit 15:8 – OVERRUN7:OVERRUN0**
  These bits reflect the **OVERRUN** status flag from the result registers for A/D channel 7 - channel 0.
- **Bit 16 – ADINT**
  This bit is 1 when any of the individual A/D channel DONE flags is asserted and enables ADC interrupt if any of interrupt is enabled in AD0INTEN register.
- **Bit 31:17 – RESERVED**

5. AD0INTEN (ADC0 Interrupt Enable)

- AD0INTEN is a 32-bit register.
- It allows control over which channels generate an interrupt when conversion is completed.

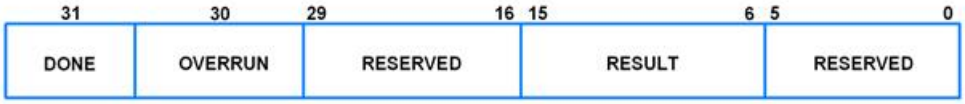


AD0INTEN (ADC0 Interrupt Enable)

- **Bit 0 – ADINTEN0**
  0 = Completion of a A/D conversion on ADC channel 0 will not generate an interrupt
  1 = Completion of a conversion on ADC channel 0 will generate an interrupt
- Remaining **ADINTEN** bits have similar description as given for **ADINTEN0**.
- **Bit 8 – ADGINTEN**
  0 = Only the individual ADC channels enabled by **ADINTEN7:0** will generate interrupts
  1 = Only the global **DONE** flag in A/D Data Register is enabled to generate an interrupt

6. AD0DR0-AD0DR7 (ADC0 Data Registers)

- These are 32-bit registers.
- They hold the result when A/D conversion is completed.
- They also include flags that indicate when a conversion has been completed and when a conversion overrun has occurred.



AD0 Data Registers Structure

- **Bit 5:0 –** RESERVED
- **Bits 15:6 –** RESULT
  When **DONE** bit is set to 1, this field contains 10-bit ADC result that has a value in the range of 0 (less than or equal to VSSA) to 1023 (greater than or equal to VREF).
- **Bit 29:16 –** RESERVED
- **Bit 30 –** Overrun
  This bit is set to 1 in burst mode if the result of one or more conversions is lost and overwritten before the conversion that produced the result in the **RESULT** bits.
  This bit is cleared by reading this register.
- **Bit 31 –** DONE
  This bit is set to 1 when an A/D conversion completes. It is cleared when this register is read.

Let's break down the steps for analog-to-digital conversion (ADC) using LPC2148 and then dive into reading analog voltage:

1. Analog-to-Digital Conversion Steps:
   - Configure ADxCR:

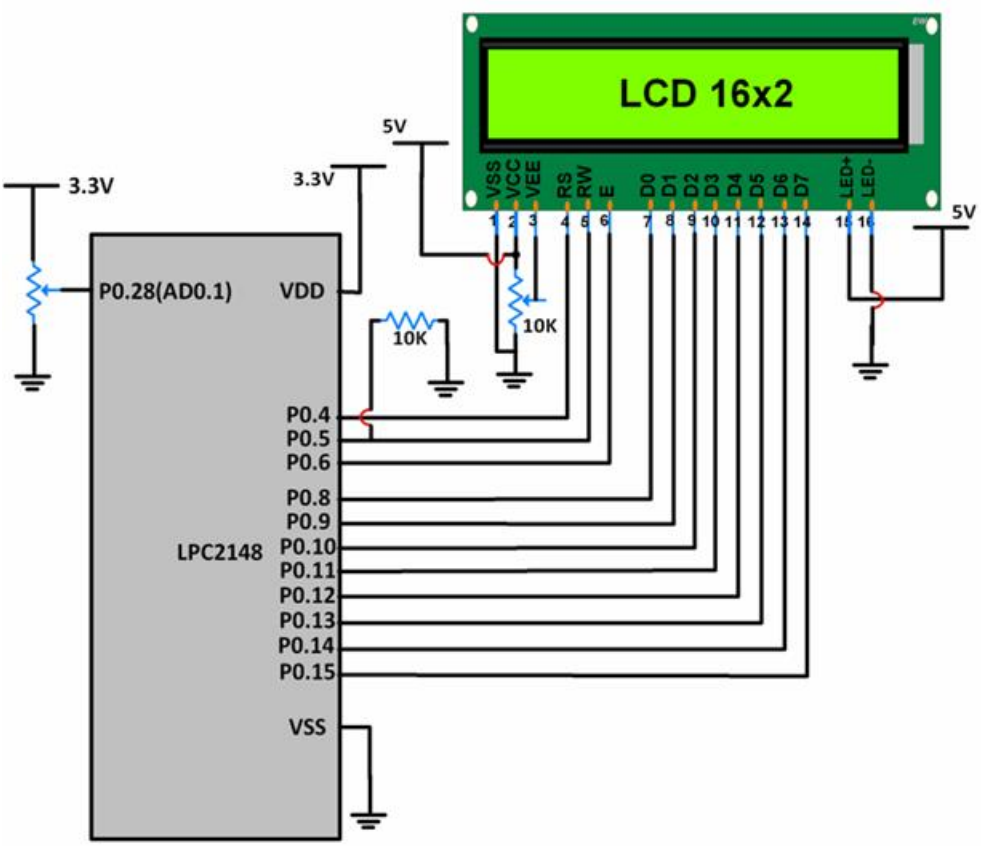
        - Set up the ADC Control Register (ADxCR) according to your application's requirements.
    - Start Conversion:
        - Write an appropriate value to the START bits in ADxCR.
        - For example, writing 001 to the START bits (bits 26:24) triggers an immediate conversion.
    - Monitor DONE Bit:
        - Keep an eye on the DONE bit (bit 31) in the corresponding ADxDRy (ADC Data Register).
        - When it changes from 0 to 1, the conversion is complete.
        - You can also monitor the DONE bit in ADGSR or the channel-specific DONE bit in the ADCxSTAT register.
    - Read the Result:
        - Retrieve the ADC result from the corresponding ADC Data Register (ADxDRy).
        - For instance, AD0DR1 contains the ADC result for channel 1 of ADC0.

Example on ADC

Reading Analog Voltage using LPC2148:

- Let's create a program to:
    - Convert the input voltage signal from AD0.1 (connected to pin P0.28) into a digital value.
    - Then, convert this digital value back to an equivalent voltage.
    - Finally, display this voltage on an LCD.
- Bonus: We can compare this displayed voltage with the actual voltage measured using a digital multimeter.

Interfacing Diagram



Potentiometer and LCD16x2 Interfacing with ESP32

Here, input signal is a DC signal which varies from 0 to 3.3V via a potentiometer.

Signal is given on P0.28. P0.28 is configured as AD0.1 using PINSEL register.

10-bit ADC result is stored in a variable and its lower 8 bits are given to P0.8-P0.15. These pins are connected to the data pins of an LCD.

P0.4,5,6 are used as RS, RW, EN pins of the LCD.

As the potentiometer is varied, we can see the variation in equivalent voltage value on the LCD.

Program for reading Analog Voltage using LPC2148

```
/*
  ADC in LPC2148(ARM7)
  http://www.electronicwings.com/arm7/lpc2148-adc-analog-to-digital-converter
*/

#include <lpc214x.h>
#include <stdint.h>
#include "LCD-16x2-8bit.h"
#include <stdio.h>
#include <string.h>

int main(void)
```

```
{
        uint32_t result;
        float voltage;
        char volt[18];
        LCD_Init();
        PINSEL1 = 0x01000000; /* P0.28 as AD0.1 */
        AD0CR = 0x00200402; /* ADC operational, 10-bits, 11 clocks for conversion */
        while(1)
        {
                AD0CR = AD0CR | (1<<24); /* Start Conversion */
                while ( !(AD0DR1 & 0x80000000) ); /* Wait till DONE */
                result = AD0DR1;
                result = (result>>6);
                result = (result & 0x000003FF);
                voltage = ( (result/1023.0) * 3.3 ); /* Convert ADC value to equivalent voltage */
                LCD_Command(0x80);
                sprintf(volt, "Voltage=%.2fV ", voltage);
                LCD_String(volt);
                memset(volt, 0, 18);
        }
}
```

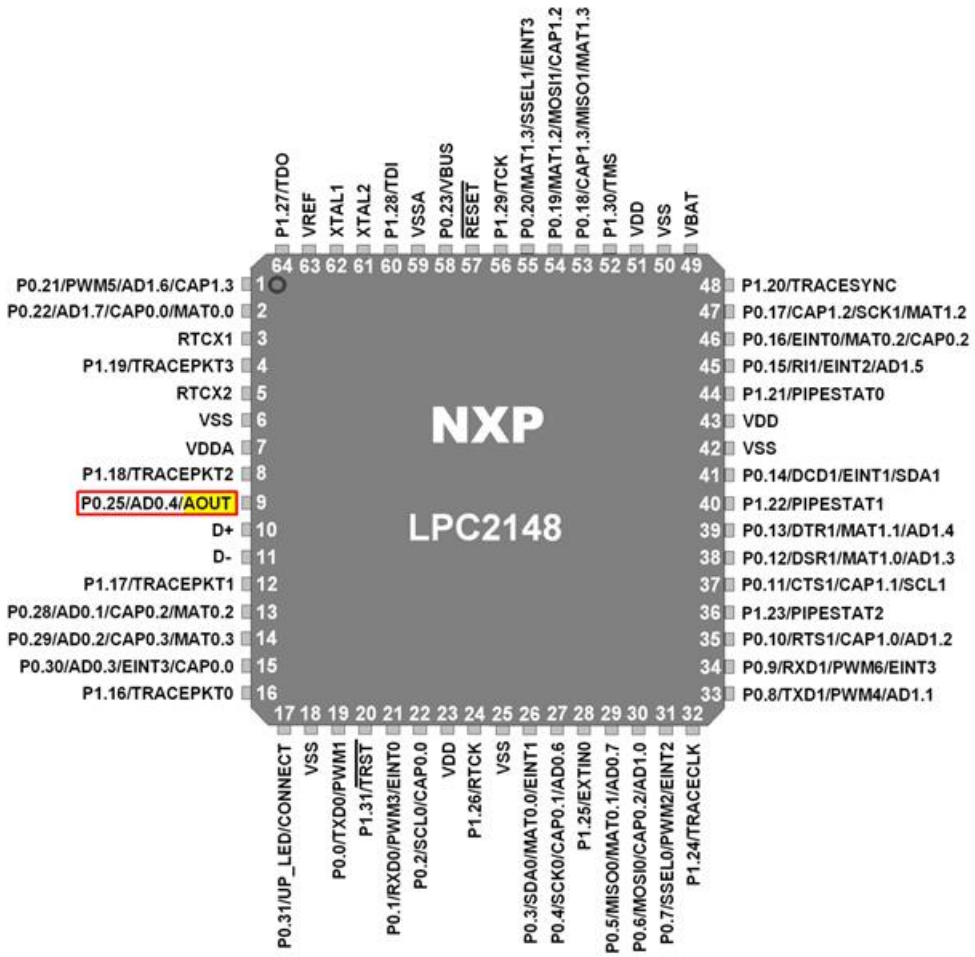
LPC2148 DAC (Digital to Analog Converter)

Introduction to DAC

Let’s explore the world of Digital-to-Analog Conversion (DAC) using LPC2148.

- **DAC Basics:**
  - Digital-to-Analog Converters (DACs) are like magical translators—they transform digital values into continuous analog signals.
  - Imagine them as turning binary whispers into analog melodies (like sine waves or triangles).
- **LPC2148’s DAC Features:**
  - The LPC2148 microcontroller boasts a **10-bit DAC** with a resistor string architecture.
  - It’s also quite versatile—it works even in **Power Down mode**.
- **Analog Output Pin (AOUT):**
  - On the chip, we have an **Analog output pin (AOUT)**.
  - This pin provides us with digital values in the form of analog output voltage.
- The Analog voltage on AOUT pin is calculated as ((**VALUE**/1024) * VREF). Hence, we can change voltage by changing **VALUE**(10-bit digital value) field in **DACR** (DAC Register).
- e.g. if we set **VALUE** = 512,
  then, we can get analog voltage on AOUT pin as ((512/1024) * VREF) = VREF/2.

LPC2148 DAC Pins



AOUT (Analog Output)

This is Analog Output pin of LPC2148 DAC peripheral where we can get Analog output voltage from digital value.

VREF (Voltage Reference)

Provides Voltage Reference for DAC.

VDDA& VSSA (Analog Power and Ground)

These are the power and ground pins for DAC. These should be same as VDD& VSS.

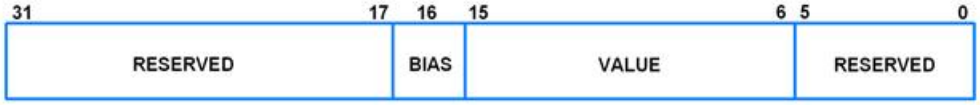
DAC Register Configuration

Let's see the Register used for DAC

DACR (DAC Register)

DACR is a 32-bit register.

It is a read-write register.



DACR (DAC Register)

Bit 5:0 – RESERVED

Bits 15:6 – VALUE

This field contains the 10-bit digital value that is to be converted in to Analog voltage. We can get Analog output voltage on AOUT pin and it is calculated with the formula (VALUE/1024) * VREF.

Bit 16 – BIAS

0 = Maximum settling time of 1µsec and maximum current is 700µA

1 = Settling time of 2.5µsec and maximum current is 350µA

Note that, the settling times are valid for a capacitance load on the AOUT pin not exceeding 100 pF. A load impedance value greater than that value will cause settling time longer than the specified time.

Bit 31:17 – RESERVED

Programming Steps

- First, configure P0.25/AOUT pin as DAC output using PINSEL Register.
- Then set settling time using BIAS bit in DACR Register.
- Now write 10-bit value (which we want to convert into analog form) in VALUE field of DACR Register.

Example

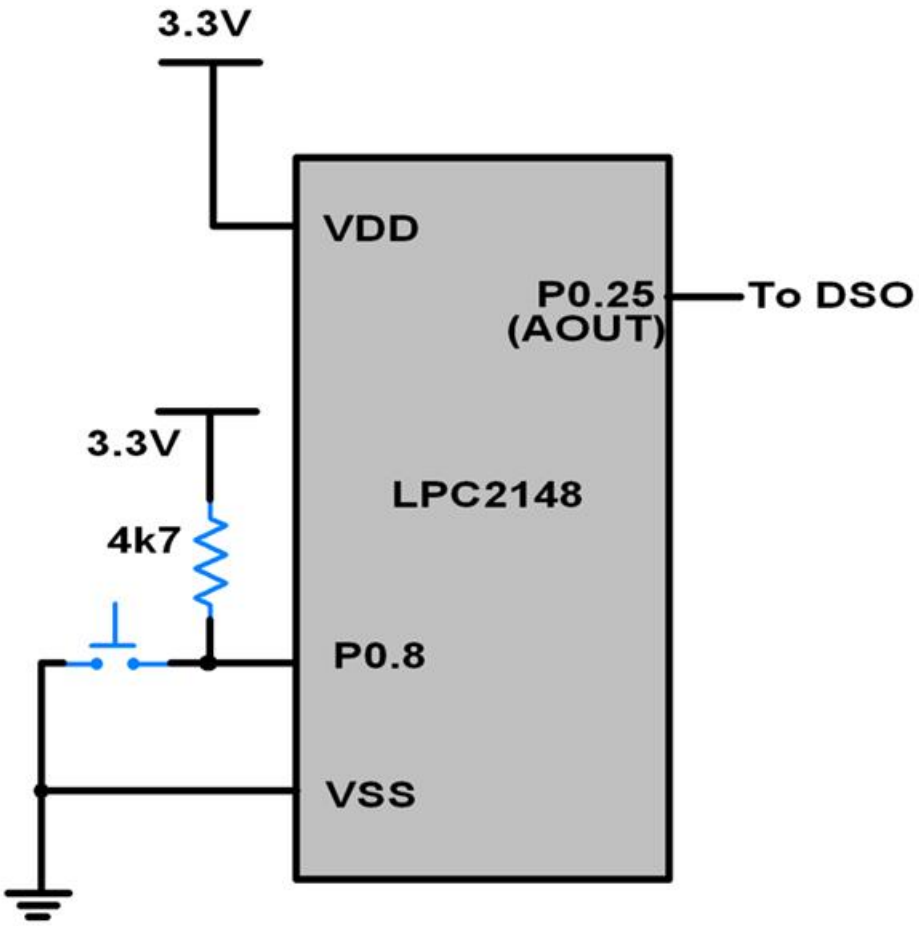
Let's embark on a waveform adventure with LPC2148's DAC!

1. **DAC Waveform Symphony:**
    - Our mission: Load the DACR value with various settings to create a palette of waves.
    - Imagine our DAC as a versatile instrument, and we'll play:
        - **Sine Wave:** Smooth and harmonious
        - **Triangular Wave:** Gradual and rhythmic
        - **Sawtooth Wave:** Edgy and descending
        - **Square Wave:** On-off, like a digital heartbeat
        - **DC Wave:** A constant voltage level.
2. **Switches as Maestros:**
    - Picture four switches:
        - P0.8 (for sine wave)
        - P0.9 (for triangular wave)
        - P0.10 (for sawtooth wave)
        - P0.11 (for square wave)
3. **Setting the Stage:**
    - Connect these pins to 3.3V through pull-up resistors.

- It’s like tuning our instruments before the performance.

4. **Oscilloscope Front Row:**
   - For the grand finale, connect the DAC output pin (P0.25) to an oscilloscope.

Interfacing Diagram



Program for LPC2148 DAC

```
/*
 DAC in LPC2148(ARM7)
 http://www.electronicwings.com/arm7/lpc2148-dac-digital-to-analog-converter
*/

#include <lpc214x.h>
#include <stdint.h>


void delay_ms(uint16_t j)
{
    uint16_t x,i;
          for(i=0;i<j;i++)
```

```
        {
    for(x=0; x<6000; x++);
        /* loop to generate 1 milisecond delay with Cclk = 60MHz */
        }
}

int main (void)
{
  uint16_t value;
        uint8_t i;
        i = 0;
        PINSEL1 = 0x00080000;  /* P0.25 as DAC output */
        IO0DIR = ( IO0DIR & 0xFFFFF0FF );
/* Input pins for switch. P0.8 sine, P0.9 triangular, P0.10 sawtooth, P0.11 square */
        uint16_t sin_wave[42] = { 512,591,665,742,808,873,926,968,998,1017,1023,1017,998,968,926,873,808,742,66
5,591,512,

  436,359,282,216,211,151,97,55,25,6,0,6,25,55,97,151,211,216,282,359,436 };
        while(1)
        {
                if ( !(IO0PIN & (1<<8)) ) /* If switch for sine wave is pressed */
                {
                                while(i !=42)
                                {
                                        value = sin_wave[i];
                                        DACR = ( (1<<16) | (value<<6) );
                                        delay_ms(1);
                                        i++;
                                }
                                i = 0;
                }
                else if ( !(IO0PIN & (1<<9)) )     /* If switch for triangular wave is pressed */
                {
                                value = 0;
                                while ( value != 1023 )
                                {
                                        DACR = ( (1<<16) | (value<<6) );
                                        value++;
                                }
                                while ( value != 0 )
```

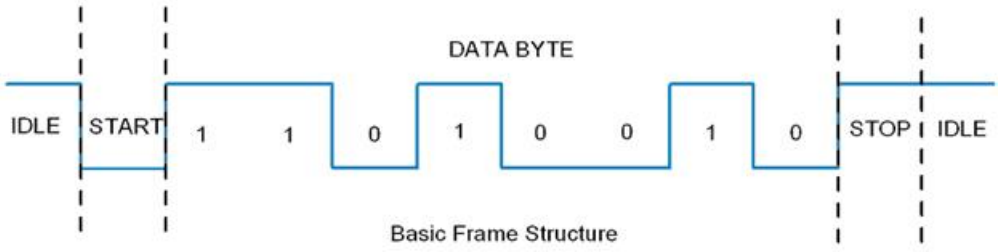
```
                        {
                                DACR = ( (1<<16) | (value<<6) );
                                value--;
                        }
                }
                else if ( !(IO0PIN & (1<<10)) )    /* If switch for sawtooth wave is pressed */
                {

                        value = 0;
                        while ( value != 1023 )
                        {
                                DACR = ( (1<<16) | (value<<6) );
                               value++;
                        }
                }
                else if ( !(IO0PIN & (1<<11)) )    /* If switch for square wave is pressed */
                {
                        value = 1023;
                        DACR = ( (1<<16) | (value<<6) );
                        delay_ms(100);
                        value = 0;
                        DACR = ( (1<<16) | (value<<6) );
                        delay_ms(100);
                }
                else      /* If no switch is pressed, 3.3V DC */
                {
                  value = 1023;
                  DACR = ( (1<<16) | (value<<6) );
                }
        }
}
```

LPC2148 UART0

Introduction

Let's delve into the scientific details of UART (Universal Asynchronous Receiver/Transmitter) and its frame structure:

1. **UART Basics:**
    - UART is a serial communication protocol where data is transmitted bit by bit in a sequential manner.
    - Asynchronous serial communication, commonly used for byte-oriented transmission, relies on UART.
2. **Frame Structure in Asynchronous Communication:**
    - When using UART, data bytes follow a specific frame structure:
        - **START Bit:**
            - Always low (logic 0).
            - Indicates the beginning of a serial communication frame.
        - **Data Bits Packet:**
            - Typically 8 bits long (although it can range from 5 to 9 bits).
            - Follows the START bit.
        - **STOP Bit:**
            - Usually one or two bits in length.
            - Sent after the data bits packet.
            - Indicates the end of the frame.
            - Always high (logic 1).



Basic Frame Structure

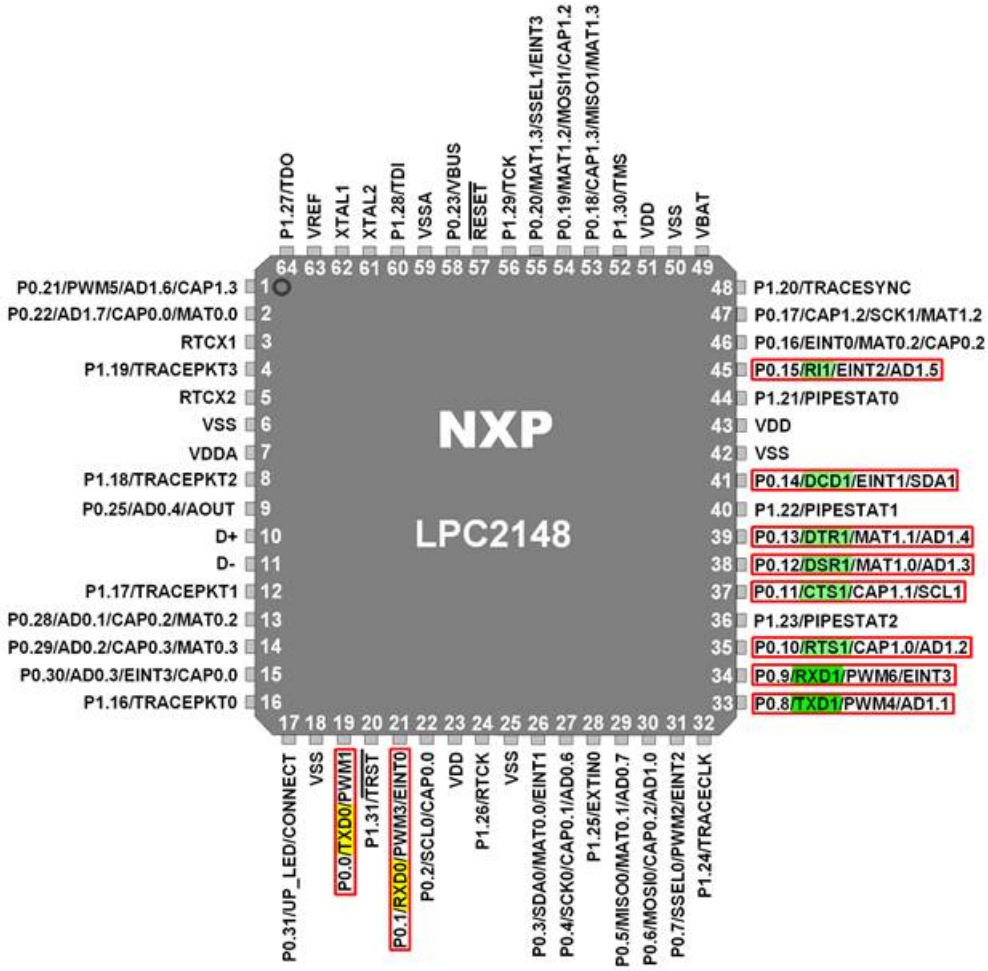
Usually, an asynchronous serial communication frame consists of a START bit (1 bit) followed by a data byte (8 bits) and then a STOP bit (1 bit), which forms a 10-bit frame as shown in the figure above. The frame can also consist of 2 STOP bits instead of a single bit, and there can also be a PARITY bit after the STOP bit.

LPC2148 UART

Let's explore the UART capabilities of the LPC2148 microcontroller.

1. **Dual UARTs:**
    - The LPC2148 comes equipped with **two inbuilt UARTs**: UART0 and UART1.
    - This means we can simultaneously connect two UART-enabled devices (think GSM modules, GPS modules, Bluetooth modules, and more) to the LPC2148.
2. **UART0 vs. UART1:**
    - Both UART0 and UART1 share many features, but there's a subtle difference:
        - **UART1** includes a **modem interface**.
        - Otherwise, they're like twins—identical in most aspects.
3. **Features of UART0:**
    - **16-Byte FIFOs:**
        - UART0 boasts both receive and transmit First-In-First-Out (FIFO) buffers, each capable of holding 16 bytes.
    - **Fractional Baud Rate Generator:**
        - Built-in and ready to handle autobauding.
    - **Software Flow Control:**
        - Controlled via the TXEN bit in the Transmit Enable Register.
4. **Features of UART1:**
    - Again, 16-byte FIFOs for both reception and transmission.
    - Fractional baud rate generator with autobauding capabilities.
    - But wait, there's more:
        - **Software and Hardware Flow Control:**
            - UART1 gives you options. You can implement flow control using either software or hardware methods.
        - **Standard Modem Interface Signals:**
            - Auto-CTS/RTS flow control is fully supported in hardware.
            - It's like having a built-in semaphore system for smooth communication.

LPC2148 UART Pins



UART0 :

TXD0 (Output pin): Serial Transmit data pin.

RXD0 (Input pin): Serial Receive data pin.

UART1 :

TXD1 (Output pin): Serial Transmit data pin.

RXD1 (Input pin): Serial Receive data pin.

RTS1 (Output pin):

Request To Send signal pin. Active low signal indicates that the UART1 would like to transmit data to the external modem.

CTS1 (Input pin):

Clear To Send signal pin. Active low signal indicates if the external modem is ready to accept transmitted data via TXD1 from the UART1.

DSR1 (Input pin):

Data Set Ready signal pin. Active low signal indicates if the external modem is ready to establish a communication link with the UART1.

DTR1 (Output pin):

Data Terminal Ready signal pin. Active low signal indicates that the UART1 is ready to establish connection with external modem.

DCD1 (Input pin):

Data Carrier Detect signal pin. Active low signal indicates if the external modem has established a communication link with the UART1 and data may be exchanged.

RI1 (Input pin):

Ring Indicator signal pin. Active low signal indicates that a telephone ringing signal has been detected by the modem.

We will be seeing how to use UART0 here. UART1 can be used in a similar way by using the corresponding registers for UART1.

UART0 Registers

1. U0RBR (UART0 Receive Buffer Register)

- It is an 8-bit read only register.
- This register contains the received data.
- It contains the “oldest” received byte in the receive FIFO.
- If the character received is less than 8 bits, the unused MSBs are padded with zeroes.
- The Divisor Latch Access Bit (DLAB) in U0LCR must be zero in order to access the U0RBR. (DLAB = 0)

U0RBR (UART0 Receive Buffer Register)

2. U0THR (UART0 Transmit Holding Register)

- It is an 8-bit write only register.
- Data to be transmitted is written to this register.
- It contains the “newest” received byte in the transmit FIFO.
- The Divisor Latch Access Bit (DLAB) in U0LCR must be zero in order to access theU0THR. (DLAB = 0)

U0THR (UART0 Transmit Holding Register)

3. U0DLL and U0DLM (UART0 Divisor Latch Registers)

- U0DLL is the Divisor Latch LSB.
- U0DLM is the Divisor Latch MSB.
- These are 8-bit read-write registers.
- UART0 Divisor Latch holds the value by which the PCLK(Peripheral Clock) will be divided. This value must be 1/16 times the desired baud rate.
- A 0x0000 value is treated like a 0x0001 value as division by zero is not allowed.
- The Divisor Latch Access Bit (DLAB) in U0LCR must be one in order to access the UART0 Divisor Latches. (DLAB = 1)

U0DLL

U0DLM

4. **U0FDR (UART0 Fractional Divider Register)**

- It is a 32-bit read write register.
- It decides the clock pre-scalar for baud rate generation.
- If fractional divider is active (i.e. DIVADDVAL>0) and DLM = 0, DLL must be greater than 3.

U0FDR (UART0 Fractional Divider Register)

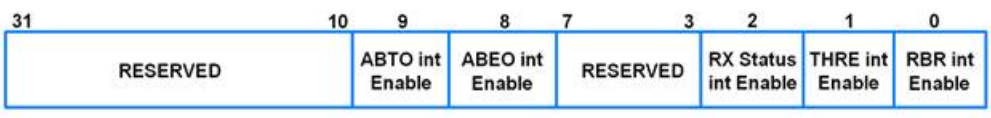
- If DIVADDVAL is 0, the fractional baudrate generator will not impact the UART0 baudrate.
- Reset value of DIVADDVAL is 0.
- MULVAL must be greater than or equal to 1 for UART0 to operate properly, regardless of whether the fractional baudrate generator is used or not.
- Reset value of MULVAL is 1.
- The formula for UART0 baudrate is given below

$$UART0Baudrate = \frac{Pclk}{16 * (256 * U0DLM + U0DLL) * (1 + \frac{DIVADDVAL}{MULVAL})}$$

- MULVAL and DIVADDVAL should have values in the range of 0 to 15. If this is not ensured, the output of the fractional divider is undefined.
- The value of the U0FDR should not be modified while transmitting/receiving data. This may result in corruption of data.

5. **U0IER (UART0 Interrupt Enable Register)**

- It is a 32-bit read-write register.
- It is used to enable UART0 interrupt sources.
- DLAB should be zero (DLAB = 0).



U0IER (UART0 Interrupt Enable Register)

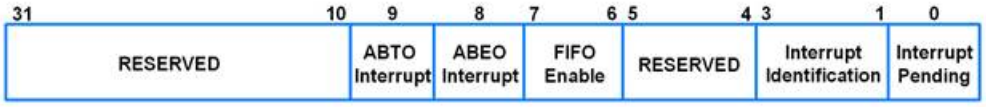
- **Bit 0 - RBR Interrupt Enable. It also controls the Character Receive Time-Out interrupt.**
  0 = Disable Receive Data Available interrupt

1 = Enable Receive Data Available interrupt

- **Bit 1 - THRE Interrupt Enable**
  0 = Disable THRE interrupt
  1 = Enable THRE interrupt
- **Bit 2 - RX Line Interrupt Enable**
  0 = Disable UART0 RX line status interrupts
  1 = EnableUART0 RX line status interrupts
- **Bit 8 - ABEO Interrupt Enable**
  0 = Disable auto-baud time-out interrupt
  1 = Enable auto-baud time-out interrupt
- **Bit 9 - ABTO Interrupt Enable**
  0 = Disable end of auto-baud interrupt
  1 = Enable the end of auto-baud interrupt

6. **U0IIR (UART0 Interrupt Identification Register)**

- It is a 32-bit read only register.



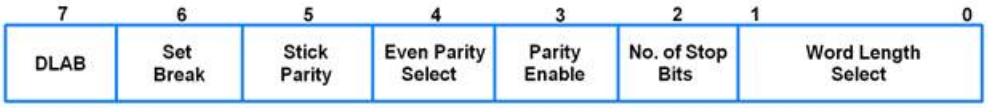
U0IIR (UART0 Interrupt Identification Register)

- It provides a status code that denotes the priority and source of a pending interrupt.
- It must be read before exiting the Interrupt Service Routine to clear the interrupt.
- **Bit 0 - Interrupt Pending**
  0 = At least one interrupt is pending
  1 = No interrupts pending
- **Bit 3:1 - Interrupt Identification**
  Identifies an interrupt corresponding to theUART0 Rx FIFO.
  011 = Receive Line Status (RLS) Interrupt
  010 = Receive Data Available (RDA) Interrupt
  110 = Character Time-out Indicator (CTI) Interrupt
  001 = THRE Interrupt
- **Bit 7:6 - FIFO Enable**These bits are equivalent to FIFO enable bit in FIFO Control Register,
  0 = If FIFOs are disabled
  1 = FIFOs are enabled
- **Bit 8 - ABEO Interrupt**
  If interrupt is enabled,
  0 = No ABEO interrupt
  1 = Auto-baud has finished successfully

- **Bit 9 - ABTO Interrupt**
  If interrupt is enabled,
  0 = No ABTO interrupt
  1 = Auto-baud has timed out

7. **U0LCR (UART0 Line Control Register)**

- It is an 8-bit read-write register.
- It determines the format of the data character that is to be transmitted or received.



U0LCR (UART0 Line Control Register)

- **Bit 1:0 - Word Length Select**
  00 = 5-bit character length
  01 = 6-bit character length
  10 = 7-bit character length
  11 = 8-bit character length
- **Bit 2 - Number of Stop Bits**
  0 = 1 stop bit
  1 = 2 stop bits
- **Bit 3 - Parity Enable**
  0 = Disable parity generation and checking
  1 = Enable parity generation and checking
- **Bit 5:4 - Parity Select**
  00 = Odd Parity
  01 = Even Parity
  10 = Forced “1” Stick Parity
  11 = Forced “0” Stick Parity
- **Bit 6 - Break Control**
  0= Disable break transmission
  1 = Enable break transmission
- **Bit 7 - Divisor Latch Access Bit (DLAB)**
  0 = Disable access to Divisor Latches
  1 = Enable access to Divisor Latches

8. U0LSR (UART0 Line Status Register)

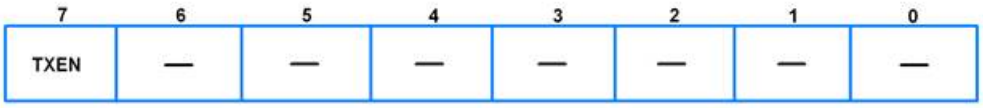
- It is an 8-bit read only register.

U0LSR (UART0 Line Status Register)

- It provides status information on UART0 RX and TX blocks.
- **Bit 0 - Receiver Data Ready**
  0 = U0RBR is empty
  1 = U0RBR contains valid data
- **Bit 1 - Overrun Error**
  0 = Overrun error status inactive
  1 = Overrun error status active
  This bit is cleared when U0LSR is read.
- **Bit 2 - Parity Error**
  0 = Parity error status inactive
  1 = Parity error status active
  This bit is cleared when U0LSR is read.
- **Bit 3 - Framing Error**
  0 = Framing error status inactive
  1 = Framing error status active
  This bit is cleared when U0LSR is read.
- **Bit 4 - Break Interrupt**
  0 = Break interrupt status inactive
  1 = Break interrupt status active
  This bit is cleared when U0LSR is read.
- **Bit 5 - Transmitter Holding Register Empty**
  0 = U0THR has valid data
  1 = U0THR empty
- **Bit 6 - Transmitter Empty**
  0 = U0THR and/or U0TSR contains valid data
  1 = U0THR and U0TSR empty
- **Bit 7 - Error in RX FIFO (RXFE)**
  0 = U0RBR contains no UART0 RX errors
  1 = U0RBR contains at least one UART0 RX error
  This bit is cleared when U0LSR is read.

9. **U0TER (UART0 Transmit Enable Register)**

- It is an 8-bit read-write register.



U0TER (UART0 Transmit Enable Register)

- The U0TER enables implementation of software flow control. When TXEn=1, UART0 transmitter will keep sending data as long as they are available. As soon as TXEn becomes 0, UART0 transmission will stop.
- Software implementing software-handshaking can clear this bit when it receives an XOFF character (DC3). Software can set this bit again when it receives an XON (DC1) character.
- **Bit 7 : TXEN**
  0 = Transmission disabled
  1 = Transmission enabled
- If this bit is cleared to 0 while a character is being sent, the transmission of that character is completed, but no further characters are sent until this bit is set again.

Programming of UART0

Let's break down the steps for initializing UART0 on the LPC2148 microcontroller:

1. Configure Pins for UART0:
    - Set P0.0 as TXD0 (transmit) and P0.1 as RXD0 (receive) by writing 01 to the corresponding bits in the PINSEL0 register.
2. DLAB and Frame Settings:
    - Using the U0LCR (UART Line Control Register):
        - Set DLAB (Divisor Latch Access Bit) to 1. This allows us to set the baud rate divisors.
        - Select an 8-bit character length and 1 stop bit.
3. Baud Rate Calculation:
    - Determine the desired baud rate (e.g., 9600).
    - Calculate the values for U0DLL and U0DLM based on the system clock frequency (PCLK).

Example,

PCLK = 15MHz. For baud rate 9600, without using fractional divider register, from the baud rate formula, we have, $$9600 = \frac{15000000}{16 * (256 * U0DLM + U0DLL)} * \frac{MulVal}{MulVal + DivAddVal}$$

On reset, MulVal = 1 and DivAddVal = 0 in the Fractional Divider Register.

Hence, $$(256xU0DLM + U0DLL) = \frac{15000000}{16 * 9600}$$

(256 x U0DLM + U0DLL) = 97.65

We can consider it to be 98 or 97. It will make the baud rate slightly less or more than 9600. This small change is tolerable. We will consider 97.

Since 97 is less than 256 and register values cannot contain fractions, we will take U0DLM = 0. This will give U0DLM = 97.

- Make DLA = 0 using U0LCR register.

```
void UART0_init(void)
{
        PINSEL0 = PINSEL0 | 0x00000005; /* Enable UART0 Rx0 and Tx0 pins of UART0 */
        U0LCR = 0x83;     /* DLAB = 1, 1 stop bit, 8-bit character length */
        U0DLM = 0x00;    /* For baud rate of 9600 with Pclk = 15MHz */
        U0DLL = 0x61;              /* We get these values of U0DLL and U0DLM from formula */
        U0LCR = 0x03;    /* DLAB = 0 */
}
```

2. **Receiving character**

- Monitor the RDR bit in U0LSR register to see if valid data is available in U0RBR register.

```
unsigned char UART0_RxChar(void) /*A function to receive a byte on UART0 */
{
        while( (U0LSR & 0x01) == 0);       /*Wait till RDR bit becomes 1 which tells that receiver contains valid data */
                return U0RBR;
}
```

3. Transmitting character

- Monitor the THRE bit in U0LSR register. When this bit becomes 1, it indicates that U0THR register is empty and the transmission is completed.

```
void UART0_TxChar(char ch) /*A function to send a byte on UART0 */
{
        U0THR = ch;
        while( (U0LSR & 0x40) == 0 );    /* Wait till THRE bit becomes
```

Assignment 1

What is an embedded system?

- A general-purpose computer system
- A specialized computer system with chip-embedded hardware
- A system that interacts with external devices only
- A system designed for high-performance computing

Which of the following statements about embedded systems is true?

- [ ] They always have real-time computing constraints.
- [ ] They are designed for one-time tasks.
- [ ] They are not cost-effective.
- [ ] They lack interaction with the environment.

What benefit does software provide in embedded systems?

- It reduces manufacturing costs.
- It enables sophisticated control.
- It eliminates the need for hardware components.
- It improves power efficiency.

Why are integrated functions often more efficient in embedded systems?

- They use external components.
- They have higher power consumption.
- They are optimized for specific tasks.
- They lack real-time capabilities.

Which application example represents an embedded system?

- Desktop computer running Windows
- Microwave oven front panel
- High-performance gaming laptop
- Supercomputer for scientific simulations

Submit Answers Clear Answers

Assignment 2

What does “IP Core” refer to in the context of ARM?

- Integrated Processor Core
- Intellectual Property Core
- Intrinsic Performance Core
- Interconnected Peripheral Core

Which type of IP core has fixed electrical properties, size, function, and shape?

- Soft IP Core
- Hard IP Core
- Flexible IP Core
- Dynamic IP Core

What is the primary difference between hard IP cores and soft IP cores?

- Hard IP cores are physically smaller.
- Soft IP cores are encrypted.
- Soft IP cores allow modification of source code.
- Hard IP cores are scalable with respect to technology.

Which ARM processor series is designed for high-performance applications with full operating system support?

- Cortex-M series
- Cortex-R series
- Cortex-A series
- SecurCore series

What does the ARM architecture describe?

- Implementation details of specific processors
- Timing information for specific processors
- Instruction set, programmer’s model, and memory map
- Electrical properties of specific processors

Which ARM processor family is suitable for cost-sensitive microcontroller applications?

- Cortex-R series
- Cortex-A series
- Cortex-M series
- SecurCore series

How many bits does an ARM7 processor have for its MCU and ALU?

- 16 bits
- 32 bits
- 64 bits
- 128 bits

Which ARM7 feature allows interfacing with 4GB of memory directly?

- 32-bit data bus
- Harvard architecture
- Von Neumann model
- 32-bit address bus

What is the purpose of ARM7's three-stage pipelining?

- To improve power efficiency
- To enhance security
- To accelerate instruction execution
- To reduce memory access latency

Which instruction set extension in ARM7 provides support for hardware debugging?

- T-Thumb Instructions
- D-Hardware Debugging
- M-Long Multiply instruction
- I-ICE Microcells

Which statement accurately describes Big Endian?

- Most significant byte is stored first in memory.
- Least significant byte is stored first in memory.
- Both significant bytes are stored simultaneously.
- It is commonly used in Intel x86 architecture.

In Little Endian, where is the last significant byte stored in memory?

- At the lowest address
- At the highest address
- In the middle of the memory
- It depends on the specific system

What is an advantage of Big Endian?

- Easier for multiplication and addition of multi-precision numbers
- Simpler to determine sign
- Better for printing data
- More efficient for division operations

How are negative numbers extended in binary for signed number representation?

- By inserting 0's ahead of the number
- By inserting 1's ahead of the number
- By reversing the order of bits
- By using a different number system

Which ARM state provides 16-bit instructions, useful for increasing code density?

- ARM State
- Thumb State
- Fast Interrupt State
- Normal Interrupt State

What does the Carry Flag © indicate in the ARM7 CPSR?

- Whether the result is zero
- Whether there is a signed overflow
- Whether there is a carry after the most significant bit
- Whether the result is negative

Which flag in the CPSR indicates whether the result of an operation is zero?

- V (Overflow Flag)
- Z (Zero Flag)
- N (Negative Flag)
- C (Carry Flag)

Submit Answers Clear Answers

Assignment 3

What is the primary advantage of assembly language?

- High-level abstractions
- Portability
- Fine-grained control over hardware
- Compatibility with modern compilers

Which language is tailored to specific tasks and can be highly efficient?

- Assembly language
- C language
- Both assembly and C
- Python

What historical context explains why assembly language was essential in the early days of computing?

- Abundance of memory
- Lack of high-level languages
- Availability of powerful compilers
- Focus on portability

What does C provide that makes it easier to learn than assembly language?

- Fine-grained control over hardware
- High-level abstractions
- Compatibility with device drivers
- Efficient memory management

Why is understanding assembly language valuable even if you don't write it?

- It helps appreciate compiler optimizations.
- It simplifies debugging in C.
- It bridges the gap between high-level and machine code.
- It allows direct manipulation of registers.

Which tool converts assembly language into machine code or object files?

- Compiler
- Linker
- Assembler
- Loader

What does the linker do in ARM7 development?

- Converts higher-level language into machine code
- Generates executable files
- Loads programs into memory
- Provides warnings for syntax errors

Submit Answers Clear Answers

Assignment 4

What does GPIO stand for?

- General-Purpose Input/Output
- Global Peripheral Interface Output
- General-Purpose Interface Operation
- Global Peripheral Input/Output

Which of the following statements about GPIO pins is TRUE?

- Their behavior is fixed and cannot be changed during runtime.
- They can only be configured as input pins.
- They can be configured as either input or output pins dynamically.
- They are only found on microcontrollers.

How many 32-bit General Purpose I/O ports does the LPC2148 microcontroller have?

- one
- two
- three
- four

Which of the following is NOT a characteristic of PORT0 on the LPC2148?

- It is a 32-bit port.
- It has 28 pins that can be configured as input or output.
- All 32 pins can be configured as general-purpose input or output.
- Pin P0.31 can only be configured as a general-purpose output.

How many pins in PORT1 are available for use as general-purpose input or output?

- 16
- 28
- 32
- 0

Which three pins in PORT0 are reserved and not available for use?

- P0.24, P0.26, and P0.27
- P0.1, P0.2, and P0.3
- P0.30, P0.31, and P0.32
- P0.8, P0.9, and P0.10

What is the primary function of an Analog-to-Digital Converter (ADC)?

- To amplify analog signals.
- To convert analog signals to digital signals.
- To filter out noise from analog signals.
- To generate analog signals from digital signals.

How many 10-bit ADCs are present in the LPC2148 microcontroller?

- one
- two
- three
- four

What does a channel in the context of an ADC represent?

- A specific output pin for the digital signal.
- A dedicated input path for an analog signal.
- A clock signal used by the ADC.
- A voltage reference for the ADC.

Which ADC in the LPC2148 has more channels?

- ADC0
- ADC1
- Both have the same number of channels.
- The number of channels varies depending on the configuration.

What is the name of the conversion technique used by the LPC2148 ADCs?

- Delta-Sigma Modulation
- Flash Conversion
- Successive Approximation
- Sample and Hold

What is the maximum clock frequency that the ADC can operate at?

- 4.5 MHz
- 10 MHz
- 20 MHz
- The clock frequency is not limited.

What is the typical voltage range for analog input to the LPC2148 ADCs?

- 0V to 5V
- 0V to 3V
- -5V to 5V
- The voltage range depends on the specific sensor used.

Submit Answers Clear Answers

References

- Marilyn Wolf , "Computers as Components: Principles of Embedded Computing System Design", 4th Edition, 2016
- Steve Furber, "ARM System-on-Chip Architecture" (2nd Edition), 2000.
- David E. Simon, "An Embedded Software Primer", 1st Edition, 1999.
- https://www.electronicwings.com/arm7 (July-2024)
- https://www.arm.com/ (July-2024)

Embedded Systems

Computer & Systems Engineering department

4th year students

2024-2025